

DART: An automated end-to-end object detection pipeline with data Diversification, open-vocabulary bounding box Annotation, pseudo-label Review, and model Training

Chen Xin^{a,b,*}, Andreas Hartel^b, Enkelejda Kasneci^a

^a Technical University of Munich, Arcisstraße 21, München, 80333, Germany

^b Liebherr-Electronics and Drives GmbH, Peter-Dornier-Straße 11, Lindau Bodensee, 88131, Germany

ARTICLE INFO

Keywords:

Open-vocabulary object detection (OVD)
Data diversification
Pseudo-label
Large multimodal model (LMM)
Stable diffusion
YOLO

ABSTRACT

Accurate real-time object detection is vital across numerous industrial applications, from safety monitoring to quality control. Traditional approaches, however, are hindered by arduous manual annotation and data collection, struggling to adapt to ever-changing environments and novel target objects. To address these limitations, this paper presents DART, an innovative automated end-to-end pipeline that revolutionizes object detection workflows from data collection to model evaluation. It eliminates the need for laborious human labeling and extensive data collection while achieving outstanding accuracy across diverse scenarios. **DART** encompasses four key stages: (1) Data Diversification using subject-driven image generation (DreamBooth with SDXL), (2) Annotation via open-vocabulary object detection (Grounding DINO) to generate bounding box and class labels, (3) Review of generated images and pseudo-labels by large multimodal models (InternVL-1.5 and GPT-4o) to guarantee credibility, and (4) Training of real-time object detectors (YOLOv8 and YOLOv10) using the verified data. We apply DART to a self-collected dataset of construction machines named Liebherr Product, which contains over 15K high-quality images across 23 categories. The current instantiation of DART significantly increases average precision (AP) from 0.064 to 0.832. Its modular design ensures easy exchangeability and extensibility, allowing for future algorithm upgrades, seamless integration of new object categories, and adaptability to customized environments without manual labeling and additional data collection. The code and dataset are released at <https://github.com/chen-xin-94/DART>.

1. Introduction

In recent years, computer vision has witnessed rapid advancements, revolutionizing various applications through innovative methodologies. A significant development is the emergence of generative AI techniques, exemplified by text-to-image Diffusion models (Podell et al., 2024; Ramesh, Dhariwal, Nichol, Chu, & Chen, 2022; Rombach, Blattmann, Lorenz, Esser, & Ommer, 2022; Saharia et al., 2022a, 2022b). They establish a robust backbone for generating high-fidelity, contextually rich images, which in turn enables other specialized frameworks (Gal, Alaluf, Atzmon, Patashnik, Bermano, Chechik, & Cohen-Or, 2022; Mou et al., 2023; Ye, Zhang, Liu, Han, & Yang, 2023) including Dream-Booth (Ruiz et al., 2023). These frameworks build upon this foundation to enhance the personalization and precision of the generated images, providing more nuanced and versatile controls. Another prominent advancement in generative AI is the large multimodal model (LMM), notably GPT-4 series (OpenAI, 2023, 2024). LMM leverages the synergy

between textual and visual data, significantly enhancing the capabilities of comprehensive semantic understanding and interaction in vision-language tasks. Concurrently, traditional computer vision areas like object detection have also experienced expedited breakthroughs. Open-vocabulary object detection (OVD) models like Grounding DINO (Liu, Zeng et al., 2023) have expanded the scope of object recognition, allowing systems to detect and classify objects beyond the constraints of predefined categories. Meanwhile, the YOLO (You Only Look Once) families (Anon, 2020; Bochkovskiy, Wang, & Liao, 2020; Jocher, 2023; Li, Li et al., 2023; Li et al., 2022; Redmon, Divvala, Girshick, & Farhadi, 2016; Redmon & Farhadi, 2017, 2018; Wang, Bochkovskiy, Liao & Mark, 2023; Wang, Chen et al., 2024; Wang, Yeh & Liao, 2024) of real-time object detection systems continue to push the Pareto frontier of speed and accuracy, achieving significant advancement through fast and continuous iterations.

Despite the above-mentioned remarkable progress in image generation, LMM, OVD, and real-time object detection, each technology

* Corresponding author at: Technical University of Munich, Arcisstraße 21, München, 80333, Germany.

E-mail addresses: chen.xin@tum.de (C. Xin), andreas.hartel@liebherr.com (A. Hartel), enkelejda.kasneci@tum.de (E. Kasneci).

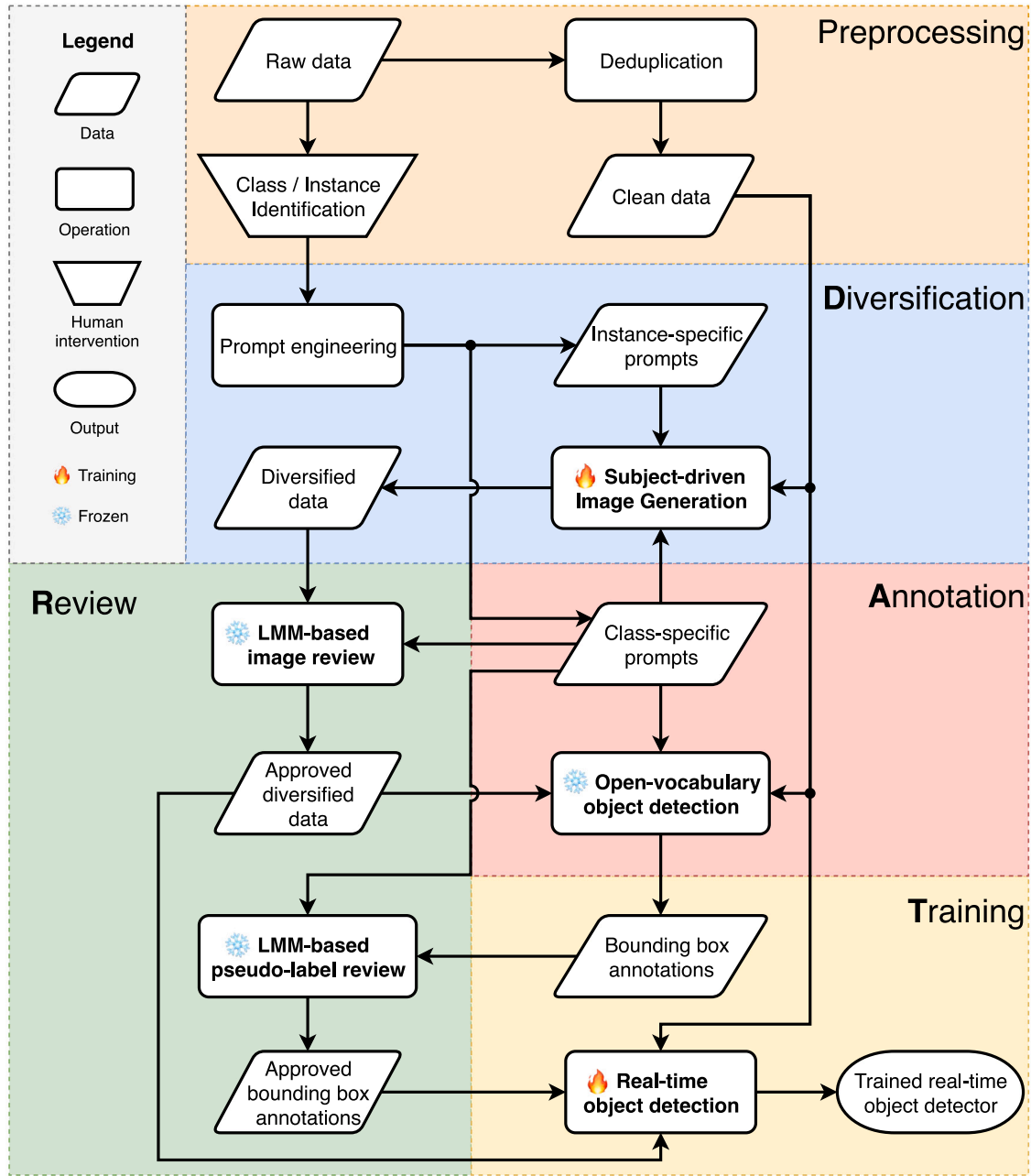


Fig. 1. DART, an automated end-to-end pipeline spanning the entire workflow of an object detection application from data collection to model evaluation. After preprocessing, DART enters the data Diversification stage, where a subject-driven image generation module (Dreambooth with SDXL) creates diversified images. The Annotation stage (red) utilizes both approved diversified data and original clean data to produce bounding boxes through open-vocabulary object detection (Grounding DINO). These pseudo-labels are then Reviewed by another large multimodal model (GPT-4o) and subsequently serve as ground truth for the Training of real-time object detectors (YOLO).

still has its prominent strengths and weaknesses. Image generation produces high-quality images but can sometimes not be photorealistic. LMMs excel in semantic understanding of images but struggle with directly handling numerical data. OVD can detect a wide range of object categories but is often too slow. In contrast, real-time object detection is fast but limited to predefined categories. If the strengths of these four technologies can be effectively combined and integrated, it will undoubtedly result in a significantly improved outcome. The proposed DART pipeline (Fig. 1) aims to achieve this complementary combination of image generation, LMM, OVD, and real-time object detection, leveraging their strengths and compensating for their weaknesses to create a comprehensive and efficient object detection solution from data collection to model evaluation.

Ideally, an object detection application should be capable of detecting any target object both quickly and accurately in diverse environments. However, current technologies do not allow for the simultaneous achievement of open-vocabulary detection, real-time capability, high accuracy, and robustness. The most common approach in the industry is to use real-time object detection to meet the essential requirement for speed while manually collecting and annotating as much data as possible to approach the other three goals (Cho & Kim, 2023; Dong, Jiang, Hu, Du, & Pan, 2024; Gao, Zhang, Geng, Tang, & Bhatti, 2024; Xu, Jha, Mehadi & Mandal, 2024). Despite the availability of numerous public datasets, they offer limited assistance in this regard because a majority of specific objects are not covered by public datasets. For instance, LVIS (Gupta, Dollar, & Girshick, 2019), one of the most diversified object detection datasets, only includes

little more than 1000 entry-level object categories, which is inadequate for many industrial applications. OVD models, on the contrary, are designed to alleviate the object coverage and manual labeling issue. However, they are excessively demanding in terms of computation and memory, making them unsuitable for real-time and on-device applications. On the other hand, the latest image generation tools, such as Stable Diffusion models, are promising alternatives to the inefficient and costly manual collection of more images of the same target object from different scenarios due to their ability for diverse image generation. However, these models lack the ability to mimic the appearance of a specific subject given by visual prompts. Customized frameworks such as DreamBooth could be applied on top for subject-driven image generation instead to address this issue. The generated images can then serve as inputs for OVD to produce bounding boxes, acting as pseudo-labels for training downstream real-time object detection models. However, the results from generative AI can sometimes be overly fanciful, and the quality of bounding boxes produced by OVD can be difficult to guarantee. This necessitates a filtering mechanism to ensure the quality of both generated images and pseudo-labels for the downstream training. This task was previously thought to be exclusive to humans, but now LMMs are well-suited for this task. Although falling short in performing number-related tasks, such as generating bounding box coordinates directly from an input image, they excel in understanding the semantic context of images. As a result, LMM can review the quality of generated images and annotations by solving a designed vision-language task in a Visual Question Answering (VQA) manner. To summarize, the foundational advancements in four areas of computer vision collectively form the backbone of the proposed automated end-to-end object detection pipeline **DART** to fulfill the four objectives of an ideal object detection application. Specifically, **DART** utilizes subject-driven image generation frameworks for data Diversification to enhance robustness, OVD for ground truth bounding box Annotation to extend object categories without human efforts, LMM for Reviewing generated images and bounding boxes to ensure accuracy and explainability, and Trained real-time object detector as the final output model.

To prove the effectiveness of the **DART** pipeline for industrial object detection applications and further exploit its potential, we collect a dataset from the internal database of Liebherr, a German-Swiss multinational equipment manufacturer. The collected dataset comprises images of Liebherr products covering a wide variety of heavy machinery such as construction machines, earthmoving equipment, deep foundation machines, mining machines, various types of cranes, material handling machines, and more. We apply dataset-specific preprocessing techniques to the raw images to obtain a clean and refined dataset named Liebherr Products (LP).

In this paper, we instantiate the **DART** using the current best algorithms for core modules and utilize the pipeline on the LP dataset. Specifically, we use DreamBooth (Ruiz et al., 2023) with Stable Diffusion XL (SDXL) (Podell et al., 2024) for data diversification, Grounding DINO (Liu, Zeng et al., 2023) for bounding box annotation, GPT-4o (OpenAI, 2024) for pseudo-label review, InternVL-1.5 (Chen, Wang et al., 2024) for generated image review, and finally YOLOv8 (Jocher, 2023) and YOLOv10 (Wang, Chen et al., 2024) families as the final object detector to be deployed. This instantiation results in an exceptional real-time object detector capable of accurately identifying and localizing all object categories with an average precision (AP) of 0.832. Without using **DART**, the result is a disastrous 0.064. The dramatic difference between object detection with and without **DART** is vividly illustrated in Fig. 13 and Fig. C.15 through qualitative comparisons. Quantitatively, we also demonstrate the effectiveness of all four major components of **DART** by showing the continuous improvement in detection performance with the incremental addition of each block. Notably, all four major blocks in the **DART** pipeline are fully automated, completely eliminating the need for human intervention. This starkly contrasts traditional supervised training methods that require

extensive human efforts in labeling and data collection. Additionally, we design the pipeline to be easily interchangeable and extendable, facilitating the adoption of more advanced algorithms in the future and enabling the seamless integration of new data and object categories with minimal human effort.

In summary, our major contributions are as follows:

1. We propose an automated end-to-end object detection pipeline named **DART** (Fig. 1). It covers the entire workflow of an object detection application, from data collection to model evaluation. Specifically, it utilizes subject-driven image generation for data Diversification, open-vocabulary object detection (OVD) for ground truth bounding box Annotation, large multimodal model (LMM) for Reviewing generated images and pseudo-labels, and Trained real-time object detector as the final output.
2. We instantiate the core module (bold text in Fig. 1) of the four major stages in the **DART** pipeline with the current best options available. Specifically, we
 - Customize DreamBooth with Stable Diffusion XL (SDXL) to generate images of the same instances but with different postures and backgrounds for data diversification,
 - apply Grounding DINO to both collected and generated images for bounding box annotation as pseudo-labels,
 - Adopt GPT-4o and InternVL-1.5 to review the quality of pseudo-labels and photorealism of generated images to determine their suitability as training data.
 - And finally train models from YOLOv8 and YOLOv10 families as the final real-time object detector.
3. We collect a dataset, named Liebherr Products (LP), of more than 15 K images covering 23 categories of construction machines from Liebherr products. We apply the instantiated **DART** pipeline on the LP dataset and obtain a highly capable YOLOv8n that can accurately identify and localize all object categories with an AP of 0.832 and a latency of 2.47 ms on a T4 GPU without any effort of human labeling. In contrast, an AP of only 0.064 is achieved without the **DART** pipeline.
4. The four key phases in the **DART** pipeline are fully automated, eliminating the need for human intervention, especially in annotation and extra data collection. Additionally, we employ a modular design in **DART** to ensure easy exchangeability and extensibility. This facilitates a smooth transition to more advanced future algorithms, seamless integration of new object categories, and adaptability to environments with customized requirements with minimal human intervention.
5. We conduct numerous experiments to demonstrate the effectiveness of **DART** by showing the continuous improvement in detection performance with the incremental addition of each of the four major components and finding the optimal tradeoff within each block.

2. Related work

In the related work section, we will focus on four areas corresponding to the **DART** pipeline's core stages in order: subject-driven image generation, open-vocabulary object detection, and large multimodal models, real-time object detection. By discussing progress in these fields, we aim to justify our current implementation of state-of-the-art methods and explore the potential for further enhancements by incorporating more advanced modules in the future.

2.1. Subject-driven image generation

Text-to-image diffusion models (Podell et al., 2024; Ramesh et al., 2022; Rombach et al., 2022; Saharia et al., 2022a, 2022b) have swiftly advanced in recent years, establishing themselves as powerful tools in

generative AI. These models generate high-quality images from textual descriptions through iterative denoising processes, forming the foundation for subject-driven text-to-image generation. Given only a few (typically 3–5) casually captured images of a specific subject, subject-driven image generation aims to produce customized images with high detail fidelity and variations guided by text prompts. One typical approach is to fine-tune a special prompt token to describe the concept of the specific subject. Textual Inversion (Gal et al., 2022) shows evidence that a single-word embedding is sufficient for capturing unique concepts. Recent methods introduce additional modules to the subject-driven image generation framework and only update the newly added modules during training while keeping the pre-trained text-to-image frozen. T2I-Adapter (Mou et al., 2023) aligns pre-trained T2I models with external control signals extracted from trainable CNNs to enhance controllability in text-to-image generation without greatly affecting the original generation ability. IP-Adapter (Ye et al., 2023) decouples the cross-attention mechanism by separating the cross-attention layers for text features and image features to achieve image prompt capability for the pre-trained text-to-image diffusion models. InstantID (Wang, Bai et al., 2024) follow the path of decoupled cross-attention and leverage a pre-trained face model to detect and extract face ID embedding from the reference facial image to generate ID-preserving images. Although these fine-tuning methods are quick and simple, the generated images often fail to completely resemble the given subject. Therefore, we adopt a full fine-tuning framework, DreamBooth, which directly fine-tunes both the text-to-image backbone with a few images of a specific subject and the token embedding, enabling the generation of diverse high-fidelity images of the specific subjects. To maximize the diversity and fidelity of the generations, we select the powerful SDXL as the backbone for the DreamBooth framework. We also improve the training process to maximize the potential of Dreambooth with SDXL and design diversified prompts to generate new images with diversified contexts.

2.2. Open-vocabulary object detection

Open-vocabulary object detection (OVD) has become a cutting-edge approach in modern object detection. It aims to detect any objects given their corresponding text prompts beyond predefined categories. Recent groundbreaking works in OVD include ViLD (Gu, Lin, Kuo, & Cui, 2022) which distills CLIP (Radford et al., 2021) embeddings to cropped image embeddings from region proposal network (RPN) (Ren, He, Girshick, & Sun, 2015), as well as GLIP (Li*, Zhang*, Zhang*, Yang, Li, Zhong, Wang, Yuan, Zhang, Hwang, Chang, & Gao, 2022) where object detection is reformulated as a grounding task by aligning bounding boxes to phrases using early fusion. Moreover, Grounding DINO (Liu, Zeng et al., 2023) extends the idea of cross-modality fusion to both feature extraction and decoder layers. It excels in zero-shot scenarios, demonstrating an impressive generalization ability to detect unseen objects. Nevertheless, one major issue of OVDs is that they are too computation- and memory-intensive to meet the needs of real-time and on-device applications. This perspective holds true even for the latest lightweight OVDs such as YOLO-world (Cheng et al., 2024). Therefore, instead of directly using OVD to solve our tasks, the proposed DART pipeline leverages OVD to generate ground truth as training data for downstream real-time object detectors. Combined with the LMM-based bounding box review (see Section 2.3 and Section 3.4), our approach results in a faster downstream detector and alleviates the speed limitations inherent in OVD.

2.3. Large multimodal model

Recent advancements in large multimodal models (LMMs) have been characterized by the release of several groundbreaking proprietary systems, including GPT-4v (OpenAI, 2023), Claude 3 (Anthropic, 2024), and Gemini (Google, 2024). These models have collectively set a new standard for vision-language understanding, demonstrating

unprecedented capabilities in combining visual inputs with advanced language processing. Recently, GPT-4o (OpenAI, 2024) further extends these capabilities with enhanced contextual awareness and multimodal reasoning, pushing the boundaries of LMM by achieving state-of-the-art performance across a wide range of vision-language tasks. Meanwhile, the open-source community has also been actively advancing in this domain. Flamingo (Alayrac et al., 2022) utilizes visual and language inputs as prompts, demonstrating exceptional few-shot performance in VQA. LLaVA series (Liu, Li, Li, & Lee, 2024; Liu, Li, Wu, Lee & Jae, 2023; Xu et al., 2024) and MiniGPT-4 (Zhu, Chen, Shen, Li, & Elhoseiny, 2023) introduce visual instruction tuning to improve the instruction-following ability of LMMs. InternVL family (Chen, Wang et al., 2024; Chen, Wu et al., 2024) scales up vision foundation models and trains on a curated high-quality dataset to achieve comparable results to GPT-4v among various multimodal benchmarks. Although open-source models have proven adequate for basic semantic understanding tasks such as image review (Lee & Zhai, 2023; Li, Wong et al., 2023), proprietary models, especially GPT-4o, still demonstrate superior performance in handling intricate visual data and sophisticated multimodal interactions, especially when additional visual hints are applied (Yang, Zhang et al., 2023). However, they both struggle with tasks requiring numerical outputs, such as directly generating bounding box coordinates (Yang, Li et al., 2023). Therefore, our DART pipeline only leverages GPT-4o/InternVL-1.5 to judge whether a model-generated output is suitable as pseudo-labels/input data for the downstream real-time object detection training while leaving the specialized task of bounding box annotation to OVD. Additionally, to better handle the review task, we craft specialized prompts tailored to the characteristics of the two LMMs, resulting in excellent outcomes.

2.4. Real-time object detection

Real-time object detection aims to accurately classify and locate objects with low latency, which is essential in numerous industrial applications. This area has witnessed significant advancements in recent years, primarily driven by the iterative development of state-of-the-art YOLO models. Early development of YOLO from v1 to v3 (Redmon et al., 2016; Redmon & Farhadi, 2017, 2018) determined the optimal segmentation of the real-time object detection network structure: backbone, neck, and head. YOLOv4 (Bochkovskiy et al., 2020) and YOLOv5 (Anon, 2020) changed the backbone to CSPNet (Wang et al., 2020) and introduced a series of best practices in computer vision regarding training strategy, post-processing, as well as plugin modules. Based on these improvements, YOLOv6 (Li, Li et al., 2023) and YOLOv7 (Wang et al., 2023) are further fine-tuned in label assignment strategy (Feng, Zhong, Gao, Scott, & Huang, 2021) and loss functions (Gevorgyan, 2022; Rezaatofghi et al., 2019; Zheng et al., 2020), and they utilize Repblock (Ding et al., 2021) and E-ELAN (Wang, Liao, & Yeh, 2022) as the core of their backbone respectively. YOLOv8 (Jocher, 2023) introduces the C2f building block for effective feature extraction and fusion, along with a comprehensive set of integrated tools, solidifying its position as the go-to choice for real-time object detection. YOLOv9 (Wang et al., 2024) proposes programmable gradient information (PGI) to refine the training process and a new lightweight network architecture – Generalized Efficient Layer Aggregation Network (GELAN). Finally, the concurrent work YOLOv10 (Wang, Chen et al., 2024) presents a training strategy without Non-Maximum Suppression (NMS) and polishes the mode architecture with holistic efficiency-accuracy-driven design, contributing to its state-of-the-art real-time object detection capabilities. Considering YOLOv8's ease of use and YOLOv10's optimal tradeoff between speed and accuracy, this work employs both the YOLOv8 and YOLOv10 families as the real-time object detectors of choice. Although we have selected the best YOLO models, their reliance on predefined object categories restricts their applicability in open scenarios. Therefore, real-time object detection only serves as the last stage in DART, depending on the other three stages to provide high-quality training data.

3. Methods

The proposed method, DART, is an automated end-to-end object detection pipeline that spans the entire workflow of an object detection application from data collection to model evaluation. As illustrated in Fig. 1, DART starts with dataset preprocessing, followed by four main stages: data Diversification through subject-driven image generation, bounding box Annotation via OVD, LMM-based Review of generated images and pseudo-labels, as well as real-time object detector Training. The core module (bold text in Fig. 1) of each phase is instantiated with state-of-the-art techniques, including the framework DreamBooth with SDXL for subject-driven image generation, the application of Grounding DINO for OVD, the adoption of GPT-4o and InternVL-1.5 for generation quality review, and the training of YOLOv8 and YOLOv10 models for real-time object detection. The following sections provide a detailed breakdown of each component of the proposed DART pipeline.

3.1. Dataset preprocessing

The data preprocessing phase, illustrated by the orange block in Fig. 1, involves identifying classes and instances from raw data as input for prompt engineering and deduplication to obtain clean data.

Class identification. The raw dataset is sourced from Liebherr's internal digital system, where images of Liebherr's products are initially roughly categorized in the database. By refining these preliminary classifications with detailed product descriptions, we establish 23 accurately defined classes, as detailed in Section 4.1. We find the precise definition of the class name essential because it highly affects the performance of the following data diversification phase since ambiguous class definition leads to inferior quality of generated data of that class. As a result, each image in the raw dataset is assigned at least one precise class name based on its primary object, and this broad assignment will be used as one of the text prompts for bounding box annotation, as described in Section 3.3.

Instance identification. For images containing a single object, we tag them at the instance level using distinctive metadata information, such as product numbers. For each class, we randomly selected 2–5 instances and manually curated approximately 10 images for each instance (see Fig. A.14). These images will be further utilized for data diversification (Section 3.2). Note that tagging with instance labels is a straightforward and easy task since we only need 20–50 images in total for each class.

Prompt engineering. Class names for images are the cornerstone for prompt construction for image generation (Section 3.2), OVD (Section 3.3), and LMM-based review (Section 3.4). The constructed class-specific prompts vary slightly for different tasks. Instance tags, however, will only serve as the backbone for instance-specific prompts for training SDXL models under the DreamBooth framework. Specific prompt construction rules will be introduced in the corresponding sections mentioned above and further detailed in Appendix B. The precise text prompts for data diversification, annotation, and review stage are listed in Table B.9, Table B.11, and Table B.12, respectively.

Deduplication. The initial raw dataset contains many duplicated images. We employ perceptual hashing (pHash) (Klinger & Starkweather, 2008) for preliminary deduplication. However, cash tends to misidentify similar images as duplicates, for example, consecutive shots of the same scene from slightly different angles. These near duplicates are manually retained in the dataset but are ensured to appear only in the training set.

Upon completing these steps, we obtain a clean dataset, dubbed Liebherr Products (LP). We also acquire instance-specific and class-specific prompts for the following phases. It should be noted that all human interventions within the DART framework conclude here, and the subsequent four phases of DART are fully automated.

3.2. Data diversification based on DreamBooth with stable diffusion XL

Data diversification and augmentation in DART are deeply rooted in the latest image-generation techniques. One of the most popular ones is Stable Diffusion, which represents a series of latent diffusion models that iteratively denoises random Gaussian noise in the latent space of a pre-trained autoencoder using a U-Net architecture (Ronneberger, Fischer, & Brox, 2015) conditioned on textual prompts. Stable Diffusion XL (SDXL) introduces several architectural improvements over previous versions of Stable Diffusion, including a larger U-Net backbone, more powerful text encoders, and novel conditioning schemes to enhance the visual fidelity of generated samples, resulting in significantly improved performance. However, SDXL takes only text prompts and, therefore, cannot mimic the appearance of a specific subject given by visual prompts, which is essentially our goal for the data diversification stage. DreamBooth addresses this issue by fine-tuning a pre-trained SDXL model using a few images of a specific subject as ground truth. The key idea is to bind a unique textual identifier with the visual subject, allowing the model to synthesize novel photorealistic images of the subject in diverse scenes and poses with the help of diversified text prompts. The framework of DreamBooth with SDXL is illustrated in Fig. 2. Without loss of generality, we showcase the class “articulated dump truck” and one of its instances, “TA230”.

During training, a pre-trained SDXL model Ω_θ with frozen weights θ is first applied to generate a bunch of class images $\mathbf{x}_{\text{cls}} = \Omega_\theta(\epsilon', \mathbf{c}_{\text{cls}})$, given initial noises $\epsilon' \sim \mathcal{N}(0, I)$ and a conditioning vector $\mathbf{c}_{\text{cls}} = \Gamma_{\theta'}(\mathbf{P}_{\text{cls}})$ generated by a text encoder Γ with frozen weights θ' using a class-specific text prompt $\mathbf{P}_{\text{cls}}$, which takes the form “a photo of a {class_name}”. Then, the same SDXL model is fine-tuned by setting θ and θ' to be trainable. In the actual implementation, we do not train the original parameters of the SDXL model and its text encoder directly but adopt LoRA (Low-Rank Adaptation) (Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, & Chen, 2022) instead which freezes the pre-trained model weights and injects trainable low-rank matrices into the transformer layers' weight matrices to enable efficient adaptation with minimal additional parameters. Here, we slightly abuse the notation of θ and θ' to also represent LoRA parameters.

The loss of the fine-tuning process is composed of two parts. The first part is the standard reconstruction loss L_{rec} for diffusion model training. Specifically, it is a squared error (SE) loss between a ground truth instance image $\mathbf{x}$ and a denoised version of it, predicted by the model Ω_θ at a specific diffusion timestep $t \sim \mathcal{U}[0, 1]$:

$$L_{\text{rec}} = w_t (\Omega_\theta(\alpha_t \mathbf{x} + \sigma_t \epsilon, \mathbf{c}_{\text{ins}}) - \mathbf{x})^2 \quad (1)$$

where α_t , and σ_t are functions of timestep t that control the noise schedule of the diffusion process. As for w_t in Eq. (1), we adopt Min-SNR weighting strategy (Hang et al., 2023) to avoid the inefficiency and instability caused by conflicts among timesteps. Specifically, $w_t = \min(\alpha_t^2/\sigma_t^2, \gamma)$, where γ is a hyperparameter to avoid the model focusing too much on small noise levels. Note that the encoded instance-specific prompt $\mathbf{c}_{\text{ins}}$ is used for the prediction of the ground truth instance. In contrast to class-specific prompts $\mathbf{P}_{\text{cls}}$ that only provide a general description of a class, instance-specific prompts $\mathbf{P}_{\text{ins}}$ should include both a general class description for better image quality and a unique identifier to build the correspondence to the given visual instance. Therefore, $\mathbf{P}_{\text{ins}}$ utilizes the extracted instance name (usually the product number) wrapped in angle brackets as the identifier (e.g. “<TA230>” in Fig. 2) followed by the original class name.

The second component of the overall loss is the class-specific prior preservation loss L_{pr} which is essentially the same SE loss but treats the class images $\mathbf{x}_{\text{cls}}$ as ground truth instead. It is introduced to mitigate the overfitting issue of the standard fine-tuning. With both reconstruction loss L_{rec} and class-specific prior preservation loss L_{pr} , DreamBooth encourages diversity in the generated images while preserving the key

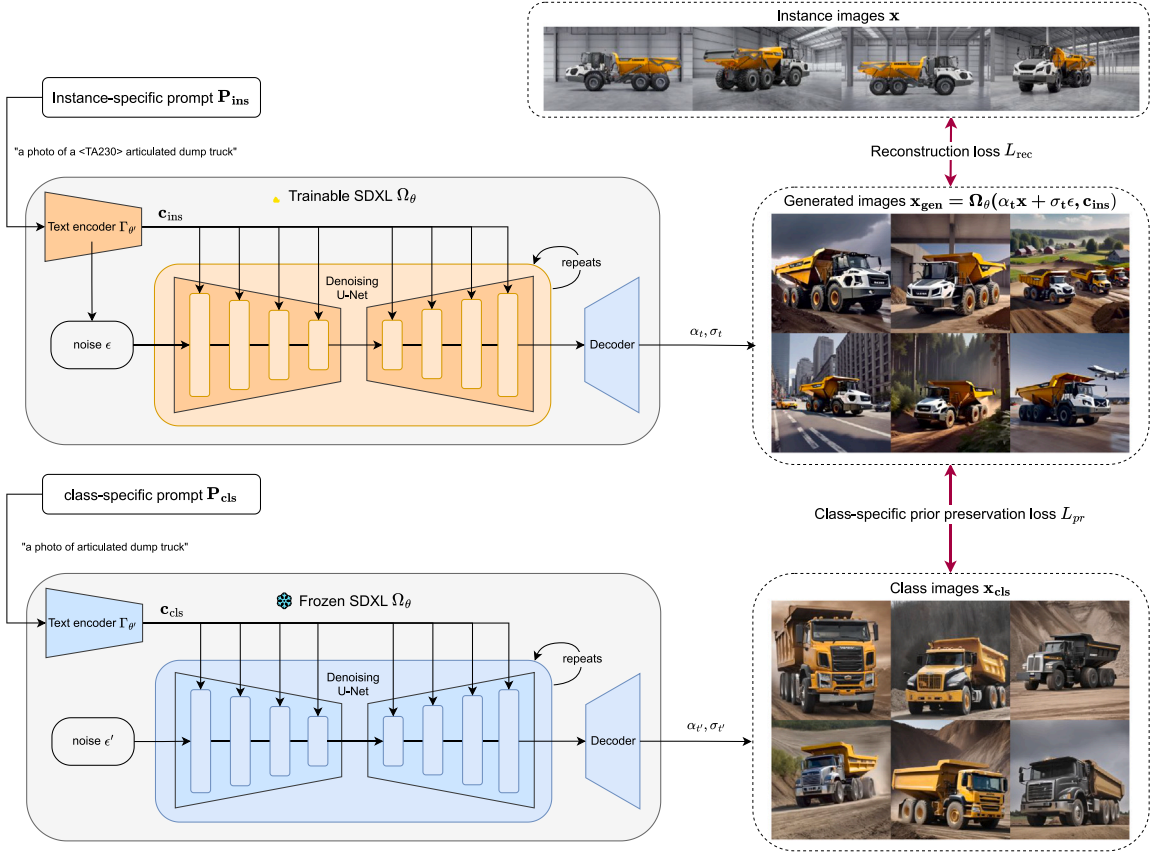


Fig. 2. DreamBooth with SDXL. Initially, class images are generated by a frozen SDXL model using a class-specific prompt (e.g., "a photo of articulated dump truck"). The model is then fine-tuned with an instance-specific prompt (e.g., "a photo of a <TA230> articulated dump truck"), optimizing both reconstruction and class-specific prior preservation losses. Source: Modified from Ruiz et al. (2023).

visual features of the subject. The total loss L can be expressed as follows:

$$\begin{aligned}
 L &= L_{rec} + \lambda L_{pr} \\
 &= w_t (\Omega_\theta(\alpha_t x + \sigma_t \epsilon, c_{ins}) - x)^2 \\
 &\quad + \lambda w_{t'} (\Omega_\theta(\alpha_{t'} x_{cls} + \sigma_{t'} \epsilon', c_{cls}) - x_{cls})^2
 \end{aligned} \quad (2)$$

We fine-tune a specific SDXL model with the DreamBooth framework for each instance. Diversified text prompts can then guide the trained SDXL models to depict each specific instance in novel contexts. We carefully design 67 text prompts in total that cover a wide and diverse range of scenarios, weather, poses, and quantities. Please refer to Table B.9 for the list of prompts used during inference for data generation. Note that to achieve our goal of diversification, we sampled a wide range of prompts covering various scenarios. As we will see in Section 4.6 and Appendix C, DART's data diversification stage effectively follows the scenarios and requirements mentioned in the text, demonstrating excellent adaptability to customized environments with minimal human intervention. At the end of the data diversification block, we obtain diversified data ready to be annotated.

3.3. Open-vocabulary bounding box annotation via groundingdino

Grounding DINO is an open-vocabulary object detector that marries a transformer-based detector DINO (Podell et al., 2024) with grounded pre-training, which can draw bounding boxes for arbitrary objects given inputs as plain text. The key to Grounding DINO's open-set capability is the tight fusion of language and vision modalities, achieved through three main components: a feature enhancer, a language-guided query selection, and a cross-modality decoder. The feature enhancer fuses language and visual features by interleaving text-to-image and

image-to-text cross-attention based on features extracted from text and image backbones in the early stage. The language-guided query selection uses language information (text features) to guide the selection of detection queries from image features, and the cross-modality decoder extends standard cross attention to two layers using image and text features as keys and values right before model outputs.

In this study, we leverage Grounding DINO for the bounding box annotation of target classes. An intuitive approach is to concatenate class names with delimiters (',' for Grounding DINO) into one text prompt as input to the model. However, we find that the model performance decreases exponentially as the length of the text grows. Thus, we turn to a strategy of incrementally adding and replacing text prompts to ensure that we ensure higher scores for ground truth accuracy.

As described in Algorithm 1, we start by using the class name that is roughly assigned during data collection (see Section 3.1) as the original prompt. To avoid misunderstanding caused by unclear semantics of the text, we design the synonym prompt using synonyms for the categories with ambiguous meanings (e.g., a handler can mean either the machine or the human operator), common aliases (e.g., dozer for bulldozer) or explicit superclasses (e.g., crane for crawler crane). A list of all synonyms can be found in Table B.10. Note that we input one synonym at a time to the model to generate bounding boxes for all synonym prompts to trade redundancy for high accuracy. The class-agnostic NMS (the last step of Algorithm 2) will finally address the redundant annotations.

To address the common phenomenon of multiple object categories in a single image and avoid the subpar performance of concatenating all classes as text prompts, we utilize the information (collected during data preprocessing) on frequently co-occurring object combinations to create another type of prompt dubbed co-occurring prompt. Unlike the original and synonym prompts, where one phrase is presented at a time,

Algorithm 1 Bounding Box Annotation

Require: Pre-trained Grounding DINO model $\mathcal{G}$ which takes given text prompt $\mathcal{T}$ and image I to generate bounding boxes $\mathcal{B}$ with confidence scores $\mathcal{S}$ and corresponding phrases $\mathcal{R}$, i.e. $\mathcal{G} : \mathcal{T}, I \rightarrow \mathcal{B}, \mathcal{S}, \mathcal{R}$,

Require: A dataset of N images $D = \{I_n\}_{n=1}^N$, a mapping of each image to its pre-defined class $\mathcal{M}_{\text{cls}} : I \rightarrow C$, a mapping of each class to its list of synonyms $\mathcal{M}_{\text{syn}} : C \rightarrow \mathcal{L}_{\text{syn}}$, and a mapping of each class to its list of common co-occurring classes $\mathcal{M}_{\text{co}} : C \rightarrow \mathcal{L}_{\text{co}}$, a prompt construction rule that takes class names and provides specific text prompt $\mathcal{P} : C \rightarrow \mathcal{T}$

Ensure: Bounding box annotations $\mathcal{B}$ for all images in D

```

 $\mathcal{B} \leftarrow []$ 
for  $I \in D$  do
   $\mathcal{B}_I \leftarrow []$   $\triangleright$  a list to store temporary annotation results for Image  $I$ 
   $c \leftarrow \mathcal{M}_{\text{cls}}(I)$   $\triangleright$  extract pre-assigned class name  $c$ 
   $\mathcal{L}_{\text{syn}} \leftarrow \mathcal{M}_{\text{syn}}(c)$   $\triangleright$  extract the list of synonyms for class  $c$ 
   $\mathcal{L}_{\text{co}} \leftarrow \mathcal{M}_{\text{co}}(c)$   $\triangleright$  extract the list of co-occurring classes for class  $c$ 
  Bounding box generation for the original prompt
   $t \leftarrow \mathcal{P}(c)$   $\triangleright$  construct original prompt
   $(b, s, r) \leftarrow \mathcal{G}(t, I)$   $\triangleright$  apply Grounding DINO
   $\mathcal{B}_I \leftarrow \mathcal{B}_I \cup (b, s, r)$   $\triangleright$  Append results
  Bounding box generation for the co-occurring prompt
  if  $\mathcal{L}_{\text{co}} \neq \emptyset$  then  $\triangleright$  if there're co-occurring classes for class  $c$ 
     $t_{\text{co}} \leftarrow \mathcal{P}(\mathcal{L}_{\text{co}}, c)$   $\triangleright$  construct co-occurring prompt using original class names
     $(b, s, r) \leftarrow \mathcal{G}(t_{\text{co}}, I)$   $\triangleright$  apply Grounding DINO
     $\mathcal{B}_I \leftarrow \mathcal{B}_I \cup (b, s, r)$   $\triangleright$  Append results
  end if
  Bounding box generation for synonym prompts
  if  $\mathcal{L}_{\text{syn}} \neq \emptyset$  then  $\triangleright$  if there're synonyms
    for  $c_{\text{syn}} \in \mathcal{L}_{\text{syn}}$  do  $\triangleright$  Loop through all synonyms for class  $c$ 
       $t_{\text{syn}} \leftarrow \mathcal{P}(c_{\text{syn}})$   $\triangleright$  construct synonym prompt
       $(b, s, r) \leftarrow \mathcal{G}(t_{\text{syn}}, I)$   $\triangleright$  apply Grounding DINO
       $\mathcal{B}_I \leftarrow \mathcal{B}_I \cup (b, s, r)$   $\triangleright$  Append results
    if  $\mathcal{L}_{\text{co}} \neq []$  then  $\triangleright$  if there're co-occurring classes for class  $c$ 
       $t'_{\text{co}} \leftarrow \mathcal{P}(\mathcal{L}_{\text{co}}, c_{\text{syn}})$   $\triangleright$  construct co-occurring prompt using the synonym
       $(b, s, r) \leftarrow \mathcal{G}(t'_{\text{co}}, I)$   $\triangleright$  apply Grounding DINO
       $\mathcal{B}_I \leftarrow \mathcal{B}_I \cup (b, s, r)$   $\triangleright$  Append results
    end if
  end for
  end if
   $\mathcal{B} \leftarrow \mathcal{B} \cup \mathcal{B}_I$   $\triangleright$  Append all annotation results for Image  $I$ 
end for
return  $\mathcal{B}$ 

```

the co-occurring prompt concatenates the names of objects commonly appearing together with delimiters into a single text. For instance, three classes of the LP dataset, “mining trucks”, “mining excavators”, and “mining bulldozers”, often appear simultaneously but rarely with other classes. Thus, images categorized into these three classes during preprocessing are given a co-occurring prompt that combines the names of the corresponding classes. This construction rule is necessary because Grounding DINO tends to produce identical boxes but different class labels when the separately given text prompt is semantically similar, which is often the case for co-occurring classes. This issue is significantly alleviated when these semantically similar texts are input into Grounding DINO as one text prompt. However, the ultimate solution to this problem is the class-agnostic NMS, which will be discussed in the next paragraph. Additionally, we apply the same rule for constructing co-occurring prompts for classes with synonyms, iterating through all combinations of these synonyms.

Algorithm 2 Filtering and NMS

Require: A dataset of N images $D = \{I_n\}_{n=1}^N$, the non-maximum-suppression (NMS) function F , bounding box annotations results consisting of generated box coordinates b , the confidence score s , and the corresponding phrase r stored as a 3-tuple for all M annotations returned from Algorithm 1, i.e. $\mathcal{B} = \{(b, s, r)_m\}_{m=1}^M$

Ensure: Final annotations $\mathcal{B}_{\text{final}}$ for all images in D

```

 $\mathcal{B}_{\text{final}} \leftarrow []$ 
for  $I \in D$  do
   $\mathcal{B}_I^{\text{raw}} \leftarrow$  a list of each annotation tuple  $(b, s, r)$  in  $\mathcal{B}$  that belongs to Image  $I$ 
   $\mathcal{B}_I \leftarrow$  a list of each annotation tuple in  $\mathcal{B}_I^{\text{raw}}$  but with its original class name  $(b, s, c)$ 
  Filtering
   $\mathcal{B}_{\text{filtered}} \leftarrow []$ 
  if  $\text{length}(\mathcal{B}_I) == 1$  then  $\triangleright$  for images with single annotation
     $\mathcal{B}_{\text{filtered}} \leftarrow \mathcal{B}_{\text{filtered}} \cup \mathcal{B}_I$   $\triangleright$  take the single annotation
  else
    for  $(b, s, r) \in \mathcal{B}_I$  do  $\triangleright$  for images with multiple annotations
      if  $s \geq 0.5$  then  $\triangleright$  extract only annotations with score  $\geq 0.5$ 
         $\mathcal{B}_{\text{filtered}} \leftarrow \mathcal{B}_{\text{filtered}} \cup (b, s, r)$   $\triangleright$  Append results
      end if
    end for
  end if
  Non-Maximum Suppression (NMS)
   $\mathcal{B}, \mathcal{S} \leftarrow$  all  $b$  and  $s$  in  $\mathcal{B}_{\text{filtered}}$   $\triangleright$  extract boxes with scores
   $\mathcal{B}_{\text{nms}} \leftarrow F(\mathcal{B}, \mathcal{S})$   $\triangleright$  apply NMS
   $\mathcal{B}_{\text{final}} \leftarrow \mathcal{B}_{\text{final}} \cup \mathcal{B}_{\text{nms}}$   $\triangleright$  Append final annotation results for Image  $I$ 
end for
return  $\mathcal{B}_{\text{final}}$ 

```

After getting the output bounding boxes and their corresponding confidence scores from original, synonym, and co-occurring prompts for all images, we apply filtering and non-maximum suppression (NMS) to keep only the best bounding boxes. As illustrated in Algorithm 2, we first sift out the ones with confidence scores bigger than 0.5 unless there is only one box for an image, in which case we take the one regardless. Then, we transform labels derived from various text prompts back to their original class names from synonyms and finally adopt class-agnostic NMS to address the issue of confusing similar labels and keep only the most feasible annotations. The remaining bounding boxes with their given class labels and scores are the candidate pseudo-labels to be reviewed.

3.4. LMM-based review of pseudo-labels and image photorealism using InternVL-1.5 and GPT-4o

LMM leverages the synergy between textual and visual data to perform tasks that require cross-modality comprehension, such as visual question answering (VQA). In the VQA setup, questions with hints and requirements are first summarized as text prompts and then further processed by a (preferably transformer-based Vaswani et al., 2017) language model. The extracted text features are combined with encoded image features (usually by a ViT Dosovitskiy et al., 2021) and then passed through another transformer to generate the text answer.

While LMMs excel in semantic understanding and comprehension, they face challenges in numeric-related tasks. For instance, directly generating bounding boxes can be problematic, as depicted in a failure case of GPT-4o in Fig. 3(a). In contrast, GPT-4o can precisely interpret the bounding box and its associated class label drawn directly on the image as visual hints, demonstrated by its correct judgments of whether the given boxes and labels are both accurate and complete, as shown in (b)–(c) in Fig. 3.



Fig. 3. Bounding box annotation and review with GPT-4o. For (a), GPT-4o is asked to directly generate the bounding box coordinates given class labels and the raw image. The returned output is drawn on the image. They are nonfunctional, as expected. In other cases, GPT-4o is requested to review the candidate pseudo-labels drawn directly on top of the image and return Yes/No answers indicating whether the labeling process is flawless. The exact text prompt used for the review task can be found in [Appendix B.3](#). For (b) and (c), only partial labels are provided, and GPT-4o correctly responds with “No”. In (d), complete labels are presented, resulting in the appropriate response of “Yes” from GPT-4o.

The proposed DART pipeline leverages GPT-4o to review the pseudo-labels generated in the bounding box annotation stage. We draw the predicted bounding boxes together with their corresponding class labels and confidence scores on top of the raw images and use these synthesized images as the visual prompts. In the text prompt, we ask GPT-4o to behave as an expert bounding box reviewer and make judgments based on three criteria: precision, recall, and fit. The corresponding three questions are:

1. Precision: Does each bounding box perfectly enclose one single target object?
2. Recall: Are all target objects localized by a bounding box?
3. Fit: Is each bounding box neither too loose nor too tight?

Besides the problem statement and requirement specification, the reminder section for the text prompt is also vital for an optimal response. We find that GPT-4 models tend to be overly strict for bounding box edges and inconsistent when dealing with occluded objects. Consequently, we summarize these common issues in text form (see [Table B.11](#) and other specific requirements for the prompt detailed in [Appendix B.3](#)) and append them at the end of the original text prompt as a reminder. This operation makes GPT-4o much less likely to make the same mistakes. Furthermore, we also utilize Chain-of-Thought (COT) (Wei et al., 2023) reasoning to enhance the model’s performance. The final text prompt used for pseudo-label review task is detailed in [Appendix B.3](#).

GPT-4o’s output is instructed to be the responses for the three key questions with brief explanations. We consider an image to be correctly

labeled only if all three questions receive positive responses. Otherwise, the image is removed from the training data.

Like bounding boxes, the generated images via Grounding DINO in [Section 3.2](#) must be validated to ensure they adhere to the requirements of downstream tasks. This verification is crucial as current SDXL models tend to produce overly imaginative images. Compared to the bounding box review based on three criteria, the image photorealism review is an easier task as it only considers photorealism. InternVL-1.5, with its robust visual comprehension, handles this task efficiently. Specifically, it is employed to conduct a preliminary review (with text prompt listed in [Table B.12](#)) of images generated by SDXL after training with DreamBooth for all selected instances, assessing their photorealism and overall quality. Only the approved images further go through the annotation phase ([Section 3.3](#)).

After GPT-4o and InternVL-1.5, only high-quality pseudo-labels and high-fidelity images are kept for real-time object detection training. We show convincing evidence in [Section 4.4](#) and [Section 4.5.3](#) that both LMM-based reviews significantly enhance the overall performance and accuracy of the final output models.

3.5. Real-time object detector training for YOLOv8 and YOLOv10

In the realm of real-time object detection, YOLO frameworks have established themselves as the dominant methodology primarily because they effectively balance computational efficiency with high detection accuracy. Currently, the most popular YOLO variant is YOLOv8. As illustrated in the figure, YOLOv8’s architecture ([Fig. 4](#)) inherits the overall framework from previous YOLO iterations, utilizing the

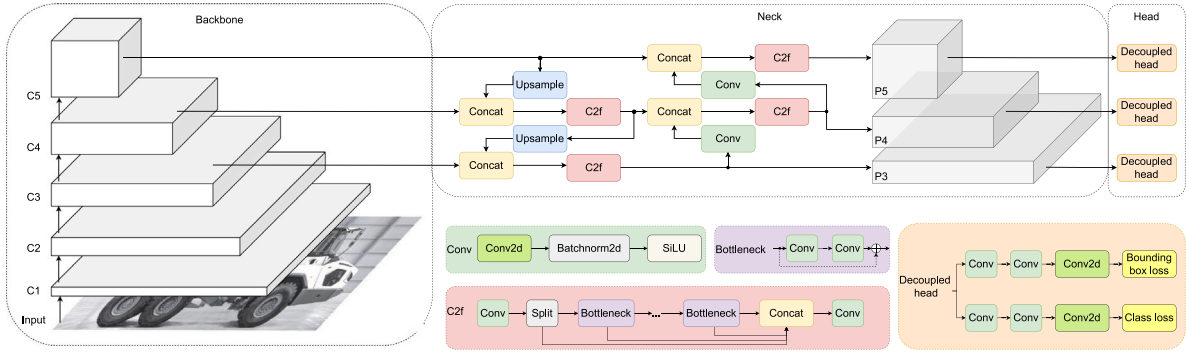


Fig. 4. Model architecture of YOLOv8. It begins with the backbone extracting multi-scale features from the input image. The last three levels of features are then passed to the "neck", where a novel C2f module is employed to effectively combine high-level semantic features with low-level spatial information. In the final stage, decoupled head modules process these features to predict bounding boxes, objectness scores, and class probabilities for each grid cell in the multi-scale feature maps. These predictions are subsequently aggregated (not shown in the figure) to produce the final detections.

Source: Modified from Jocher (2023), RangeKing (2023).

Darknet (Redmon & Farhadi, 2018) backbone, SPP (Spatial Pyramid Pooling He, Zhang, Ren, & Sun, 2014) and PAN (Path Aggregation Network Liu, Qi, Qin, Shi, & Jia, 2018) structure in the neck, and three multi-scale decoupled heads to predict bounding box coordinates and class labels. For the major block in the backbone and neck, YOLOv8 employs the C2f module, which is a faster implementation of the CSPBlock (Wang et al., 2020) featuring two convolution layers at each end, with the CSP Bottlenecks in between (depicted as the red block in Fig. 4). In terms of training strategy, the model adopts an anchor-free approach and incorporates Task Alignment Learning (TAL) (Feng et al., 2021) for ground truth assignment and Distribution Focal Loss (DFL) (Li et al., 2020) as the regression loss. These enhancements contribute to improved computational efficiency and detection accuracy. The concurrent YOLOv10 is the latest release in the YOLO family. It proposes a consistent dual assignments strategy for NMS-free training as well as a holistic efficiency-accuracy-driven model design featuring comprehensive adoption of pointwise, depthwise, and large-kernel convolutions with partial self-attention (PSA), which greatly reduces the computational overhead and enhances model capability.

With approved (Section 3.4) diversified (Section 3.2) images and their corresponding pseudo-labels (Section 3.3), training a real-time object detector becomes plain and easy. Since DART's output will be deployed on edge devices, we select and train only the nano (n) and small (s) variants of YOLOv8 and YOLOv10. These models, pre-trained on the COCO dataset (Lin et al., 2014), are fine-tuned using approved diversified data and pseudo-labels. Extensive experimentation reveals the best-performing YOLO models, considering the tradeoff between accuracy, speed, and model size (see Section 4.5.4). By demonstrating the superior performance of the YOLO models under various conditions (see Section 4.4 and Section 4.5), our experiments also underline the outstanding performance of the DART pipeline, validate the effectiveness of the DART pipeline, and pinpoint the optimal variant for each stage.

4. Experiments

4.1. Dataset

Data collection. We collect a dataset named Liebherr Products (LP) from the internal database of Liebherr, a German-Swiss multinational equipment manufacturer. During data collection, we also employ class and instance identification (in Section 3.1) for bounding box annotation (in Section 3.3) and data diversification (in Section 3.2), respectively.

Dataset composition. The LP dataset (after preprocessing) encompasses more than 15 K images of Liebherr's extensive range of heavy machinery such as construction machines, earthmoving equipment, deep foundation machines, mining machines, various types of cranes,

material handling machines, and so on. A list of the exact 23 categories can be found either as the x-axis tick labels in Fig. 5 or in the first column of Table B.10. Fig. 5 illustrates the distribution of images in each category, represented by the blue bars labeled "0:1". The unbalanced data distribution in the LP dataset makes the downstream object detection task even more challenging. However, as we will see in Fig. 9, the final YOLO models still achieve consistently good results across various classes, highlighting the effectiveness of DART. Furthermore, Fig. 5 also shows the data distribution after incorporating varying proportions of generated data, which will be elaborated on in Section 4.5.1.

Dataset split. The dataset is initially split into a training and a test set with an 80/20 distribution. To address class imbalance, stratified random sampling ensures each class is adequately represented in the test set. We allocate 20% of the training set to create a validation set, which is used for hyperparameter fine-tuning. After obtaining the optimal hyperparameter set, the model is trained on the original training with 12 K images and evaluated on the 3K images from the test set. The training set could incorporate generated data in specific experiment setups to enhance data diversification. Unless otherwise specified, the generated-to-original data ratio is maintained at the optimal proportion of 3:1, as outlined in Section 4.5.1. Notably, categories that already exhibit strong performance with only the original data are excluded from the augmentation with generated data, as detailed in Section 4.5.4. All performance metrics reported in the tables and figures of this paper are based on the test set.

4.2. Evaluation metrics

In this work, we utilize the average precision (AP) of the trained object detection models in the last stage as the evaluation metrics for the proposed DART pipeline. AP summarizes the precision-recall (PR) curve into a single value, serving as an approximation of the area under the PR curve. To compute AP, precision p and recall r are first evaluated as follows:

$$p(t) = \frac{TP}{TP + FP}, r(t) = \frac{TP}{TP + FN}, \quad (3)$$

where detections are classified as true (T) or false (F) according to their confidence scores and determined as positive (P) or negative (N) based on the Intersection over Union (IoU) threshold t .

Then a modified version of PR curve $\tilde{p}(r, t, c)$ is constructed for each class $c \in C$ given a certain IoU t :

$$\tilde{p}(r, t, c) = \max_{r' \geq r} p(r', t, c), \quad (4)$$

so that the curve will decrease monotonically instead of in a zigzag pattern.

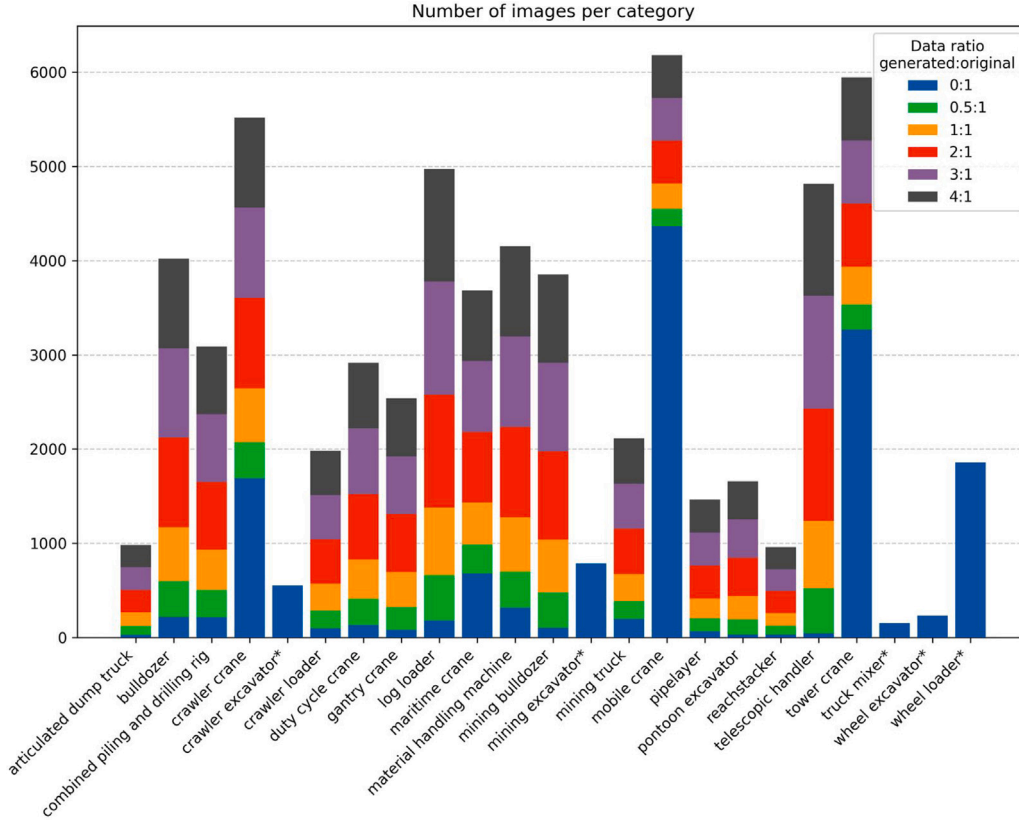


Fig. 5. Number of images per category for different ratios of generated to original data. Each color in the color bar represents the incremental data increase from the previous to the current ratio. Note that five classes (marked by *) are not considered during the data diversification phase given their already strong initial performance. See Section 4.5.4 for more details.

Following COCO (Lin et al., 2014), $\bar{p}(r, t, c)$ is interpolated and evaluated at 101 points where recall $r \in [0 : .01 : 1]$ for AP calculation. We specify the IoU thresholds of interest by adopting two commonly used AP metrics: AP_{50} and AP_{50-95} . The former measures precision at a single IoU of 50%, while the latter averages precision across 10 IoU thresholds ranging from 50% to 95% in increments of 5% ($t \in [.50 : .05 : .95]$). To summarize, AP_{50-95} can be expressed as follows:

$$AP_{50-95} = \frac{1}{10} \sum_{t \in [.50 : .05 : .95]} \left(\frac{1}{|C|} \sum_{c \in C} \left(0.01 \sum_{r \in [0 : .01 : 1]} \bar{p}(r, t, c) \right) \right) \quad (5)$$

Note that AP_{50-95} is no greater than AP_{50} since the precision decreases for a given recall with higher thresholds. For a well-trained model on diverse and complex datasets, such as LP, both metrics exhibit similar trends. These two consistent patterns will also be observed in the following tables. Consequently, the following sections will primarily focus on the AP_{50-95} . Since AP_{50-95} is also commonly abbreviated as AP, we will use these two terms interchangeably.

4.3. Implementation details

4.3.1. Implementation of DreamBooth with SDXL

We instantiate DreamBooth with SDXL as the subject-driven image generation framework for the data diversification phase. The SDXL model is preferable, as it significantly outperforms previous versions of Stable Diffusion models such as SD-1.5 (Rombach et al., 2022). We incorporate prior preservation loss to prevent overfitting and adjust its weight (λ in Eq. (2)) to 1.0. Contrary to SD-1.5, training the text encoder of SDXL has limited effects; we still opt to fine-tune it to enhance overall performance marginally. We extract 69 instances for the training of DreamBooth with SDXL in total. The minimum number of images per instance is set to 3 (as suggested by Ruiz et al., 2023),

while the majority of instances have 10 images each (see Fig. A.14). We implement dynamic training steps by adjusting the maximum training steps based on the number of images collected for the corresponding instance. 100–160 steps per image are proven to be suitable, combining with the AdamW optimizer (Loshchilov & Hutter, 2019) with a learning rate 1×10^{-4} for U-Net and 5×10^{-6} for the text encoder. The SNR-gamma is set to 5 by default. Other important hyperparameters are explained in Appendix A.1.

We fine-tune two models from the same official pre-trained checkpoint for each instance: one with slightly more steps to ensure resemblance and one with fewer for diversification. For training, a general class-specific prompt, “a photo of a {class_name}”, is employed to generate class images for prior preservation loss. As for instance-specific prompts, we use the instance name (product number if available, otherwise a descriptive word) to form the prompt as “a photo of a <{instance_name}> {class_name}”. We also explore different approaches to constructing identifiers and class names. However, our experiments reveal that subtle modifications have negligible effects on performance as long as the identifier is sufficiently distinctive and the class name is adequately precise. To meet the requirements of data diversification, we craft inference text prompts encompassing a broad spectrum of scenarios, as listed in Appendix B.1. The training processes are executed on either a single NVIDIA A100 80 GB GPU or two NVIDIA 3090 GPUs, with each configuration capable of completing the training of one instance-specific SDXL model in less than an hour. The inference process requires substantially less GPU memory and is compatible with any memory optimization techniques for SDXL.

4.3.2. Implementation of Grounding DINO

Off-the-shelf Grounding DINO base model is employed for bounding box annotation. Through extensive comparative experiments, we determined that a box threshold of 0.27 and a text threshold of 0.25

generally returns high-quality bounding boxes, resulting in only 18 images without annotations, which are consequently excluded from the pipeline. The original collected images and approved generated images are isometrically resized to 800/1333 pixels for the shorter/longer side before going through the model. As for text prompts, we follow instructions given in Section 3.3 to construct all three prompt types as separate inputs to the Grounding DINO model and apply filtering and NMS to the combined results to obtain the candidate bounding boxes to be reviewed. The annotation process is mainly carried out (including speed test as shown in Table 3) by an NVIDIA A100 80 GB GPU with FP16 for our case.

4.3.3. Implementation of LMM

We leverage GPT-4o by calling OpenAI's API for pseudo-label review. The bounding boxes and class labels are directly drawn on top of the image, and the synthesized image is compressed to 512 pixels on the longer side while keeping the aspect ratio before being specified as the vision prompt for the API. Detailed task descriptions, judging criteria, hints, suggestions, and reminders are all listed in the text prompt. The prompt construction rule is outlined in Section 3.4, and the exact content of text prompts is detailed in Appendix B.3. We employ the GPT-4o-based pseudo-label review only to images containing multiple annotations or annotations with confidence scores lower than 0.5 (after NMS in Section 3.3), which accounts for approximately 5K images since we empirically find that Grounding DINO generally provides accurate bounding boxes and labels when the confidence score exceeds 0.5. As for InternVL-1.5, we use it for a simpler task of photorealism review to filter out unrealistic images generated by each SDXL model trained by the DreamBooth framework. We run the InternVL-1.5 model locally on an NVIDIA A100 80 GB GPU with the visual prompt as a single generated image and the text prompt given in Table B.12.

4.3.4. Implementation of YOLO

As the final step of the DART pipeline, we fine-tune COCO pre-trained YOLOv8 and YOLOv10 models as the output object detector to be deployed. We follow common practice (Anon, 2020; Jocher, 2023; Li, Li et al., 2023; Li et al., 2022; Wang et al., 2023; Wang, Chen et al., 2024; Wang et al., 2024) to resize all images to 640×640 . Then, AdamW is implemented with a weight decay set to $5e-4$ as the optimizer. Next, we conduct extensive experiments focusing on learning rate and its scheduling for each model and each experiment setup. The optimal initial learning rates range from $7e-5$ to $5e-4$, with a general trend indicating that larger models and bigger datasets benefit from lower learning rates. Mosaic augmentation (Anon, 2020; Bochkovskiy et al., 2020) is adopted to enhance data diversity. Details on other critical hyperparameters are provided in Appendix A.3. Speed tests (including NMS for YOLOv8, referenced from the official codebase or paper) are conducted on an NVIDIA T4 GPU using TensorRT and FP16. The test results are reported in Table 6.

4.4. Ablation study on the overall DART pipeline

In this section, we perform an ablation study on the overall DART pipeline. We systematically analyze the contributions of each major component by evaluating the test performances of trained real-time object detection models under various configurations. Specifically, we incrementally integrate the first three stages of DART, i.e., Grounding DINO for annotation, GPT-4o for review, and DreamBooth with SDXL for data diversification, as shown in Table 1. To evaluate the impact of each newly incorporated component, we leverage the overall performance of DART's final stage as the metric. Various YOLO models with different versions and scales, including YOLOv8n, YOLOv8s, YOLOv10n, and YOLOv10s, are fine-tuned for a comprehensive assessment. We observe a consistent trend in performance increase across all four YOLO models as new components of DART are introduced. Therefore, the subsequent analyses will primarily use YOLOv8n as the

representative model. For the complete test results on the other models, please refer to Table 1.

Our baseline configuration (#0) utilizes a YOLOv8n model without any additional modules from DART, yielding the lowest performance with an AP of just 0.064. In this setup, the model is trained on one of the most abundant datasets, LVIS (Gupta et al., 2019). Despite LVIS's extensive coverage of over 1.2K categories across 164K images, it falls short of capturing all the classes present in our self-collected LP dataset. Consequently, we assign each LP's category to its closest kin in LVIS during inference, guided by similarity scores calculated from class names and descriptions by SBERT (Reimers & Gurevych, 2019). The results, however, are still disastrous, with a merely non-zero AP of 0.064, suggesting the necessity of the DART pipeline. Integrating the bounding box annotation stage of DART improves the AP to a usable level 0.771. Further incorporation of GPT-4o to rule out inferior pseudo-labels shows an incremental performance increase, resulting in a proper AP of 0.802 and thereby indicating the additional value of LMMs for the DART pipeline. The full power of DART is achieved by the final integration of data diversification. Adding data generated by SDXLs trained for each instance in a DreamBooth framework (and approved by InternVL-1.5) to the original training set boosts the final best AP to 0.832 for the YOLOv8n model, underlining the superior efficacy of the comprehensive DART pipeline.

4.5. Quantitative analysis

In this section, we conduct internal analyses for the four major stages of the proposed DART pipeline by investigating the best solutions for each key module via comprehensive experiments and ablation studies within each stage.

4.5.1. Analysis of DreamBooth with SDXL

We begin by examining the impact of incorporating extra training data generated by SDXL models trained for each instance using the DreamBooth framework. Across all configurations, we maintain a consistent test set of 3K images and augment the training set with generated data (with both images and pseudo-labels approved by LMM) in varying multiples (from 0 to 4) of the original 12K images.

Table 2 presents the results of training YOLOv8 models with different ratios of generated to original data. When the training data comprises entirely generated data (0:1 ratio), the YOLOv8n model achieves an AP of only 0.031. This outcome is expected due to two primary factors. First, our objective of data diversification drives us to intentionally describe scenarios in the text prompts during inference that are entirely different from the original images (see Appendix B.2). Secondly, the current DreamBooth with SDXL cannot produce results identical to the original objects but with slight differences. Both factors contribute to a significant disparity between the train and test data distributions. In other words, we deliberately induce an extreme data shift, leading to poor performance on the test set. Conversely, using only original data (ratio 0:1) and hereby eliminating both issues boosts the AP to 0.815.

However, as we gradually increase the amount of generated data, the AP continues to rise, reaching 0.832 for YOLOv8n at a 3:1 ratio (see also Fig. 6 for a visualization of this increase). This phenomenon is due to the influence of another opposing mechanism to data shift: introducing diversified data simultaneously enriches the training data diversity, benefiting the model's learning process. The changes in model performance at different ratios result from the interplay between data shift and enrichment mechanisms. As shown in Fig. 6, further increasing the generated data causes the data shift mechanism to slightly dominate, adversely affecting test performance.

In summary, original data is indispensable for a decent test performance, while generated data offers significant benefits, especially when at a ratio when data shift and diversification mechanisms balance. For the LP dataset, the optimal scenario involves using three times the generated data as the original data. The almost identical trends observed in both YOLOv8n and YOLOv8s models clearly illustrated in Fig. 6 corroborate this conclusion.

Table 1
Ablations on key stages of the DART pipeline.

#	Model	Grounding DINO	GPT-4o	DreamBooth	AP_{50}^{test}	AP_{50-95}^{test}
0					0.091	0.064
1	YOLOv8n	✓			0.864	0.771
2		✓	✓		0.899	0.802
3		✓	✓	✓	0.915	0.832
4	YOLOv8s	✓			0.883	0.789
5		✓	✓		0.903	0.815
6		✓	✓	✓	0.924	0.850
7	YOLOv10n	✓			0.854	0.738
8		✓	✓		0.877	0.777
9		✓	✓	✓	0.879	0.802
10	YOLOv10s	✓			0.867	0.781
11		✓	✓		0.889	0.809
12		✓	✓	✓	0.917	0.833

Table 2
Impact of training data ratios (generated:original) on YOLOv8 model performance.

Training data ratio generated:original	YOLOv8n		YOLOv8s	
	AP_{50}^{test}	AP_{50-95}^{test}	AP_{50}^{test}	AP_{50-95}^{test}
1:0	0.047	0.031	0.047	0.031
0:1	0.899	0.802	0.903	0.815
0.5:1	0.902	0.815	0.911	0.834
1:1	0.908	0.822	0.916	0.836
2:1	0.909	0.825	0.914	0.843
3:1	0.915	0.832	0.924	0.850
4:1	0.914	0.826	0.918	0.839

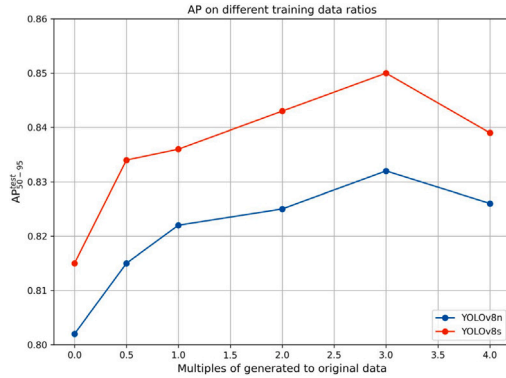


Fig. 6. Test performance of YOLOv8 models across different training data ratios (generated:original).

4.5.2. Analysis of grounding DINO

Two Grounding DINO variants, which utilize Swin tiny and base (Liu et al., 2021) as the backbone, respectively, are tested for the bounding box annotation task. It can be seen from Table 3 that using the more powerful base variant increases test performance for the YOLOv8n model, suggesting better consistency in bounding box annotations and label assignment. However, both of the Grounding DINO variants run too slow to fit the real-time requirements, even on one of the most powerful GPUs, NVIDIA A100. Consequently, we only adopt the Grounding DINO base model for all data annotation tasks for better bounding box quality despite a slightly slower speed, which is negligible for the offline labeling process.

Prompt engineering plays a vital role in the performance of Grounding DINO. In Section 3.3, we outline the construction process for three types of prompts (original, synonym, and co-occurring) and introduce approaches for merging their outputs, including filtering and NMS. In this section, we will discuss the results from these prompts and merging operations reported thoroughly in Table 4. As reported in Section 3.3, simple concatenation of all class names as a single text prompt for one image (type “all”) leads to inferior performance reflected quantitatively by the low average confidence score of 0.33 in

Table 3
Comparisons between Grounding DINO tiny and base.

Grounding DINO	Input size	Latency ^a	AP_{50}^{test} ^b	AP_{50-95}^{test} ^b
Tiny	800 × 1333	112 ms	0.858	0.745
Base	800 × 1333	161 ms	0.864	0.771

^a Latency is tested on a NVIDIA A100 80GB with FP16.

^b APs are provided by a YOLOv8n model trained on all original images annotated by Grounding DINO tiny and base, respectively.

Table 4

Quantities and average confidence scores of Grounding DINO annotations under various types of text prompts and operations. In particular, “all” means concatenating all class names as one text prompt for each image. The three prompt types that are practically applied in DART are “original,” “synonym,” and “co-occurring”. After combining the output from these three prompt types, “filtering” is applied to sift out most annotations with a confidence score < 0.5. Finally, class-agnostic non-maximum suppression “NMS” is employed to get rid of overlapping annotations. More details regarding prompt types and operations can be found in Section 3.3.

Type	# annotations	Average score
all	76913	0.33
original/co-occurring	33271	0.49
+ synonym/co-occurring	76664	0.47
+ filtering (≥0.5)	30580	0.66
+ NMS	17309	0.65

the table and qualitatively by the first row of Fig. 12. Therefore, we start with only a predefined class name as the original prompt and the co-occurring trick for multiple categories. Next, synonym replacement is applied for all text prompts, which results in more than double annotations with only a marginal change in the average confidence score. Synonym prompts are indispensable because our experiments reveal that a substantial number of remaining bounding boxes, which survive the high-probability filtering and class-agnostic NMS, are originally derived from synonyms. We ultimately achieve non-overlapping bounding boxes with an overall score of 0.65, where the average score for each category is depicted by the blue bars in Fig. 5 (note that the y-axis starts at 0.5). This ensures that, at least within the Grounding DINO framework, only high-quality bounding boxes are selected. These bounding boxes will be further examined by GPT-4o.

4.5.3. Analysis of LMM

DART utilizes GPT-4o for bounding box review. The detailed construction of visual and text prompts is thoroughly discussed in Section 3.4 and further detailed in Appendix B.3. According to the responses from the pseudo-label review by GPT-4o, we eliminate slightly less than 2K images with only disapproved bounding boxes. This results in improvements in the confidence score of bounding boxes among

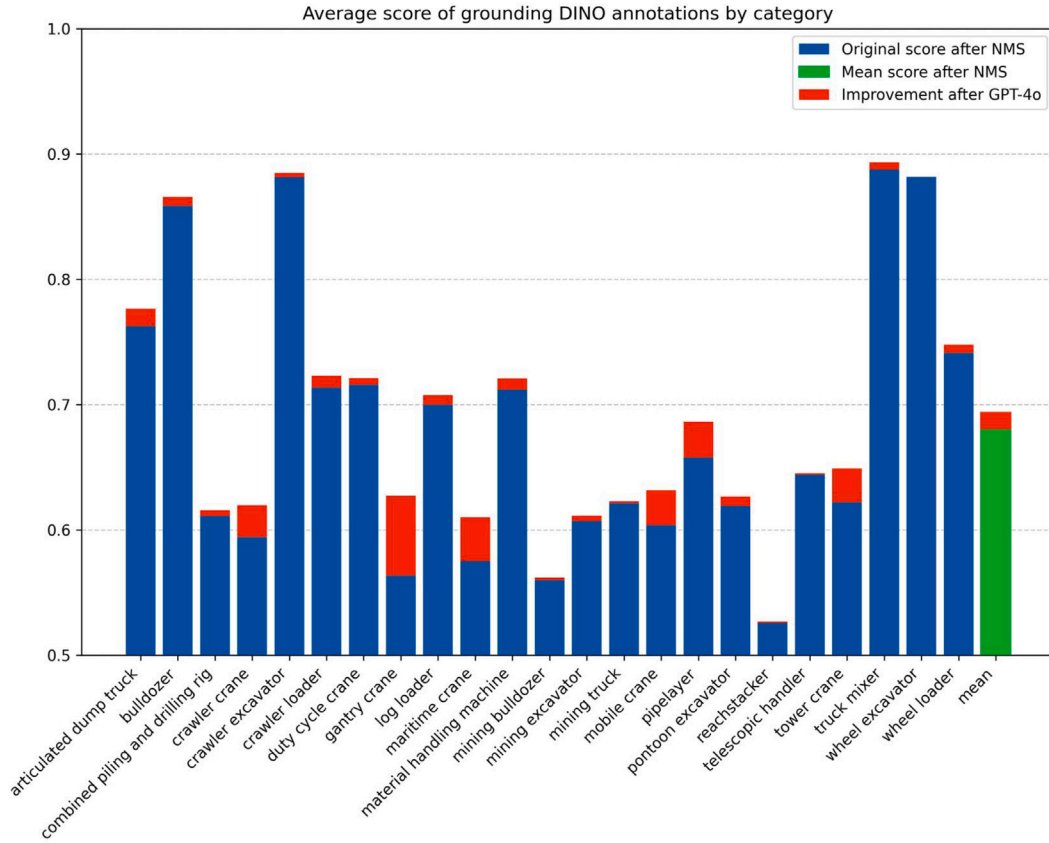


Fig. 7. Class-wise average confidence score of Grounding DINO annotations after NMS filtering (blue) and GPT-4o review (red).

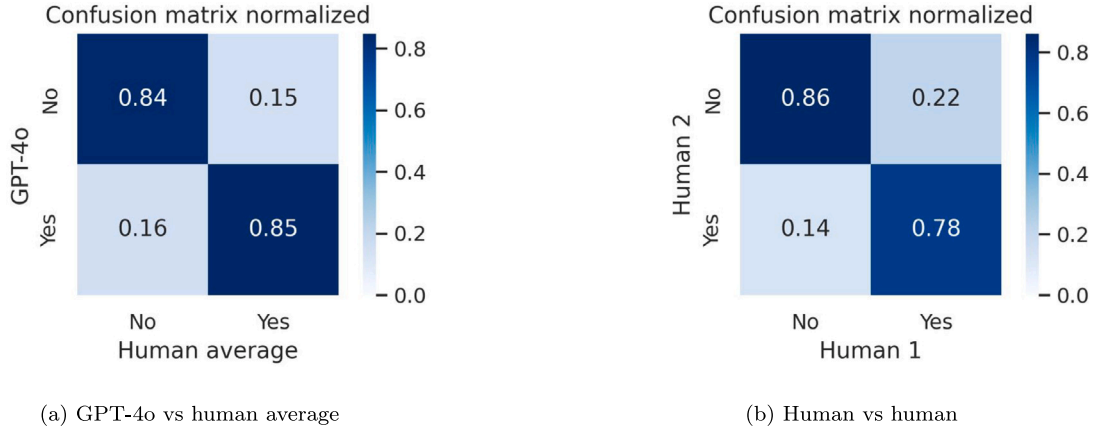


Fig. 8. Comparisons of bounding box review consistency. The left confusion matrix displays the agreement between GPT-4o and the human average, whereas the right matrix compares the annotations of two experienced human labelers. GPT-4o demonstrates human-level performance in bounding box review given the same text and visual information.

all categories as displayed by the red bars in Fig. 5, indicating the consistent judgment between GPT-4o and Grounding DINO regarding high-quality bounding box annotations.

It turns out that GPT-4o's decision is even more in line with experienced human labelers. We conduct a small-scale human evaluation experiment by providing experienced labelers with the same instructions as GPT-4o and recording their responses to 50 randomly selected images with pseudo-labels drawn on top. In Fig. 8, the left confusion matrix illustrates the results of GPT-4o vs average human performance, while the right exhibits the agreement level between two labelers for reference. Notably, GPT-4o and human labelers give consistent responses for approximately 85% of the cases, a rate comparable to the agreement between two human labelers.

Table 5
Ablations on LMM-based reviews.

GPT-4o	InternVL 1.5	AP_{50}^{test}	AP_{50-95}^{test}
✓		0.871	0.782
✓		0.909	0.818
✓	✓	0.915	0.832

Despite GPT's exceptional visual comprehension abilities, its API costs are high. Therefore, we leverage the open-source model InternVL-1.5 for the straightforward task of verifying the photorealism of generated images. More details on this task are also given in Appendix B.3.

Table 6
Comparisons of YOLO models.

Model	Input size	#Param.	FLOPs	Latency ^a	AP ^{test} ₅₀	AP ^{test} _{50–95}
YOLOv8n	640 × 640	3.2M	8.7G	2.47 ms	0.915	0.832
YOLOv8s	640 × 640	11.2M	28.6G	2.74 ms	0.924	0.850
YOLOv10n	640 × 640	2.3M	6.7G	1.84 ms	0.879	0.802
YOLOv10s	640 × 640	7.2M	21.6G	2.49 ms	0.917	0.833

^a Latency is tested on a NVIDIA Tesla T4 with TensorRT FP16, referenced from the official codebase or paper.

We design a scenario to validate the effectiveness of both LMMs by comparing the test performance of a YOLOv8n model trained with and without the images they disapproved. First, we construct a dataset using the optimal generated-to-original data ratio of 3:1 obtained in a Section 4.5.2. As indicated in the first row of Table 5, training a YOLOv8n directly on this dataset results in a test performance of AP 0.782. Subsequently, when we remove the images not approved by GPT-4o from the original dataset and retrain the model, the AP climbs up to 0.818. Note that this is already better than training without any generated data, which provides an AP of 0.802 as reported in case #2 of Table 1, exhibiting not only the clear advantages of employing GPT-4o-based filtering but also the pure benefit of data diversification. Finally, by applying InternVL-1.5 to sift through images generated by SDXL and retain the photorealistic ones, the trained model achieves the best performance with an AP of 0.832.

4.5.4. Analysis of YOLO

YOLO models are chosen as the final real-time object detector for DART. Fig. 9 depicts the class-wise test performance of two identical YOLOv8n models applied with partial (a) and full (b) DART. The first confusion matrix is derived from the YOLOv8n model, on which only the annotation and training stage of DART is employed. Note that we choose these two DART components from the outset since the pre-trained YOLO models fail catastrophically in object detection on the LP dataset as explained in Section 4.4. The realization that providing zero-shot performance comparison is meaningless already emphasizes the essential need for DART.

Without the full ability of DART, we observe several misclassifications (non-diagonal elements) and subpar true positive rates (diagonal elements), particularly in categories such as “combined pilling and drilling rig”, “crawler loader”, “log loaders”, and so on, indicating room for improvement. However, certain categories, such as crawler excavators and truck mixers, already exhibit satisfactory performance. We hypothesize that this is due to their relative prevalence compared to other categories and their inclusion in several public datasets (e.g., LVIS). This enables Grounding DINO to provide more accurate and consistent bounding boxes, as indicated by the high confidence scores of these classes in Fig. 7. Consequently, these categories do not require further data diversification. Instead, we focus on diversifying data for categories with temporarily lower performance, allocating more quota of generated images to these classes to improve their results. We name this strategy selective data diversification.

The second confusion matrix in Fig. 9 represents the results from the full DART pipeline. Obviously, the selective data diversification process described above pays off. Misclassifications are reduced across almost all categories. True positive detections for the diversified categories considerably elevate without negatively impacting the performance of the previously well-performing categories. Note that selective data diversification exacerbates the imbalance in the dataset by favoring other categories over five unchosen ones. Despite this, the model’s performance on the five underrepresented categories remains unaffected, highlighting DART’s resilience to data imbalance. This observation confirms the efficacy of selective data diversification, leading to its overall deployment for all experiments involving the addition of generated diversified data.

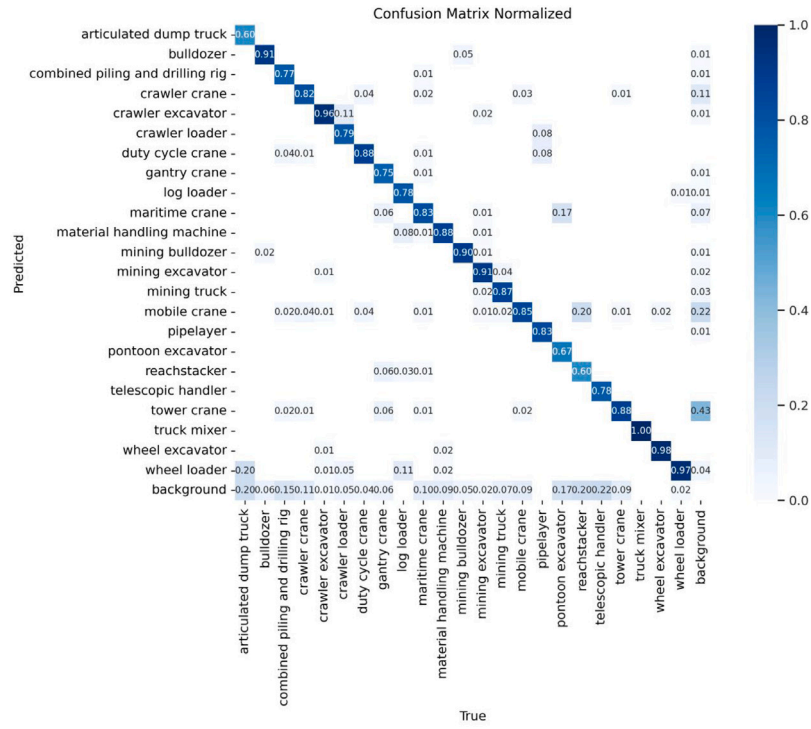
Finally, the statistics for all real-time object detectors in this study are listed in Table 6. Each YOLO variant is enhanced by the best options provided by each DART component, i.e. photorealistic generated images judged by InternVL-1.5 and mixed with original data at the optimal ratio of 3:1, bounding box annotations carried out by Grounding DINO base model, pseudo-labels approved by the LMMs, and the best hyperparameter set sifting through extensive fine-tuning experiments. The AP values are always derived from the same test set. As seen from the third column, the YOLOv10 models are designed to possess fewer parameters than the YOLOv8 model of the same category (indicated by the suffix n or s). Due to efficiency-driven model design, the YOLOv10n model achieves an AP of 0.802 in just 1.84 ms (on a T4 GPU with TensorRT FP16). This is emphasized in the left subfigure of Fig. 10, where other models fall slightly behind YOLOv10n in the AP-speed tradeoff. Increasing the number of parameters by approximately 40% and inference time by 30%, the YOLOv8n model significantly boosts the AP to 0.832. The right diagram of Fig. 10 clearly shows that YOLOv8n achieves a Pareto optimal balance between AP and model size, outperforming all other models in this regard. If the sole focus is on AP, YOLOv8s becomes the optimal choice, albeit at the cost of the highest parameter count and the slowest inference time. We reasonably speculate that larger and more complex models, such as the m/l/x variants of the YOLO series or state-of-the-art transformer-based object detectors (Wang, Dai et al., 2023; Zong, Song, & Liu, 2023) would further elevate AP. However, this contradicts DART’s goal of real-time performance on edge devices, so they are not included in this study.

In summary, the fastest model is YOLOv10n, the most accurate is YOLOv8s, but the overall best model for real-time object detection purposes is YOLOv8n, as it achieves the Pareto optimal balance between AP and model size, with a practical AP performance of 0.832. We propose the trained YOLOv8n as the final output of the DART pipeline. In contrast to the disastrous performance without DARG and the subpar results of an incomplete DART, the comprehensive DART pipeline clearly demonstrates superior efficacy in enhancing the capabilities of real-time object detection models. Compared to the catastrophic performance without DARG and the subpar results of an incomplete DART, the comprehensive DART pipeline clearly demonstrates superior efficacy in enhancing the capabilities of real-time object detection models. The fact that DART accomplishes these advancements without needing manual annotation and extra data collection further highlights its significance in effectiveness and efficiency.

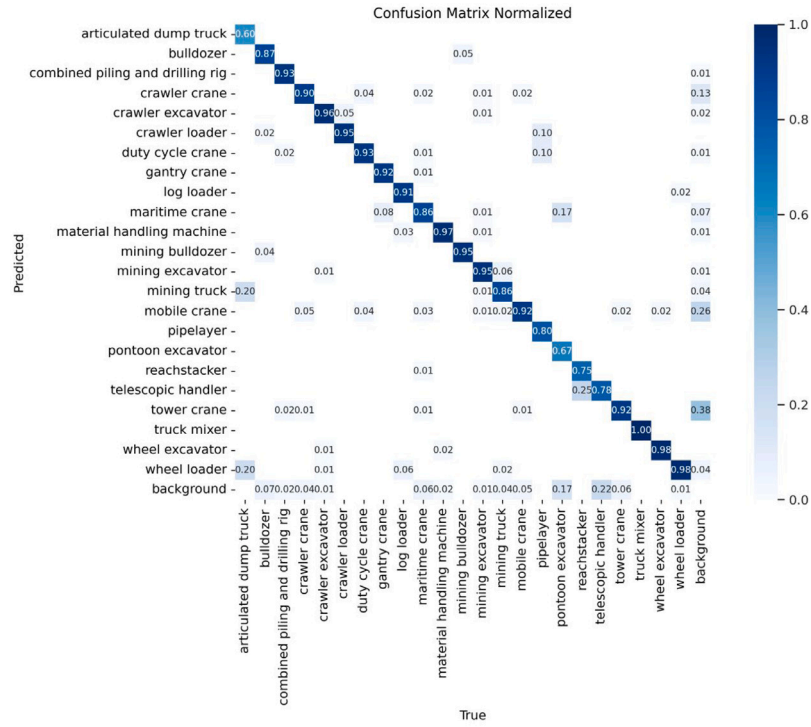
4.6. Qualitative analysis

In this section, we conduct qualitative analyses using visualization to examine the model’s performance during the data diversification, bounding box annotation, and LMM-based review stages. We focus on three specific scenarios in this section, while numerous other visualizations can be found in Appendix C.

Fig. 11 showcases a randomly selected category, “combined pilling and drilling rig”, to demonstrate the visualization of the data diversification and bounding box annotation stages. Additional illustrations of other categories can be found in Figs. C.16 and C.20. As described in Section 3, we first select some instance data and use them to train SDXL under the DreamBooth framework. The trained SDXL generates diversified data during inference with the help of designed prompts (with their IDs and descriptions shown as subtitles for each generated image on the right side; refer to Table B.9 for the corresponding text). After approval by InternVL-1.5 (omitted in the figure but showcased in Fig. C.19), pseudo-labels (directly drawn on top of each image) are obtained through Grounding Dino. As illustrated in Fig. 11, the SDXL model trained with DreamBooth accurately generated realistic objects. Guided by diversified text prompts, the resulting generated images effectively depicted the scenes described. The designed prompts used for data generation also allow for the customization of background, pose, and quantity of target objects, greatly enriching the original dataset



(a) only with DART's annotation and training



(b) with full DART

Fig. 9. Confusion matrices derived from the class-wise test performance of the same YOLOv8n model enhanced by incomplete (a) and full (b) DART pipeline.

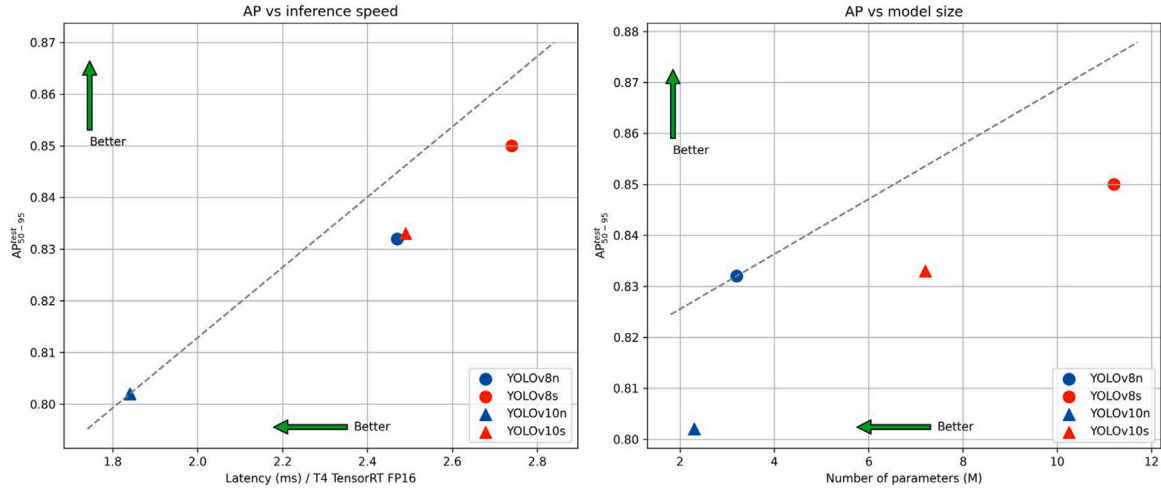


Fig. 10. Test performance comparisons of YOLOv8 and YOLOv10 models in terms of accuracy-latency (left) and accuracy-size (right) tradeoff. YOLOv10n and YOLOv8n reach the Pareto frontier in these two scenarios, AP vs. inference speed and AP vs. model size, respectively. The dotted lines in both figures are derived from nearly collinear points of the three non-optimal models and are translated to the position of the optimal model to highlight its superiority.



Fig. 11. Visualization of data diversification and bounding box annotation for the category “combined pilling and drilling rig”. Instance data on the left are utilized for training an SDXL under the DreamBooth framework. The trained SDXL generates diversified data during inference using designed prompts. The IDs and descriptions of the corresponding prompts used to generate the image are shown as the corresponding subtitles. Please refer to Table B.9 for the exact content of each text prompt. Bounding box coordinates and labels are given by Grounding DINO and are drawn directly on top of the generated images on the right side. Also see Figs. C.16 and C.20 for more illustrations of other classes.

to meet personalized requirements. The bounding box annotations are also highly precise. In summary, DART significantly increases dataset diversity and generates accurate pseudo-labels for the newly created data. These enhancements substantially improve the performance of downstream YOLO models compared to training on the original dataset alone, as shown repeatedly in Section 4.4 and Section 4.5.

Derived from the responses from GPT-4o-based pseudo-label review, Grounding DINO delivers feasible results in even 60% of hard cases with confidence scores below 0.5. Nonetheless, current state-of-the-art open-vocabulary detection (OVD) models still exhibit certain issues. As shown in Fig. 12, Grounding DINO’s performance significantly declines when the prompt is simply too long. This limitation prevents us from using fully concatenated class names as a single text prompt (which is the case for the first row in Fig. 12 and marked as “all prompt”). Instead, we must adopt a workaround involving original class names, synonyms, and co-occurring classes and their combinations as detailed in Section 3.3. Additionally, text ambiguity can pose problems. For instance, “handler” (case d in Fig. 12) can mean either an object or a person, and “gantry crane” (e) can refer to either the entire structure or individual cranes. Furthermore, OVD is certainly not immune to common challenges faced by all object detection models, including omitted target objects (f), difficulties with small objects (g), incorrect classification (h), or box coordinates (i). Fortunately, LMM can assist in identifying these errors. Our GPT-4o-based pseudo-label review detects 5 out of 6 instances of errors in the last two rows of Fig. 12, demonstrating both the necessity and efficacy of the proposed LMM-based review.

Finally, we first visually compare object detection results with and without DART in Fig. 13. The first column in the figure visualizes

ground truth (labeled by Grounding DINO and approved by GPT-4o), while the other two showcase prediction results of a YOLOv8n model trained with and without DART. Each row picks an image from the test set, which is never seen by the object detector in either scenario. Clearly, without DART’s customized diversification and annotation, the model performance remains poor even after fine-tuning on LVIS. On the contrary, introducing DART leads to a substantial performance boost, yielding almost identical results to the ground truth. This aligns with the dramatic AP increase from 0.064 to 0.832 discussed in Section 4.5.4, demonstrating the necessity and effectiveness of DART.

In the last row of Fig. 13, we demonstrate an even more intriguing benefit of DART: the final YOLO model of DART can capture missed objects during the bounding box annotation process. We argue that the YOLO model benefits from the diversity of poses and backgrounds for the target object category provided by DART, allowing it to better learn the target object’s concept. Conversely, Grounding DINO is limited by the extensive range of its pre-trained public dataset, making it sometimes less effective in specialized detection tasks. This suggests a new approach for the annotation stage: fine-tuning OVD with manually labeled target objects before using it for bounding box generation. We leave this idea for future exploration. On the other hand, the fact that training a tiny real-time object detection model under the DART framework without any manual labeling and extra data collection leads to rectifying errors made by the state-of-the-art OVD model highlights the effectiveness of the DART pipeline. Despite the imperfections of existing OVD models, DART’s robust framework compensates for their shortcomings.

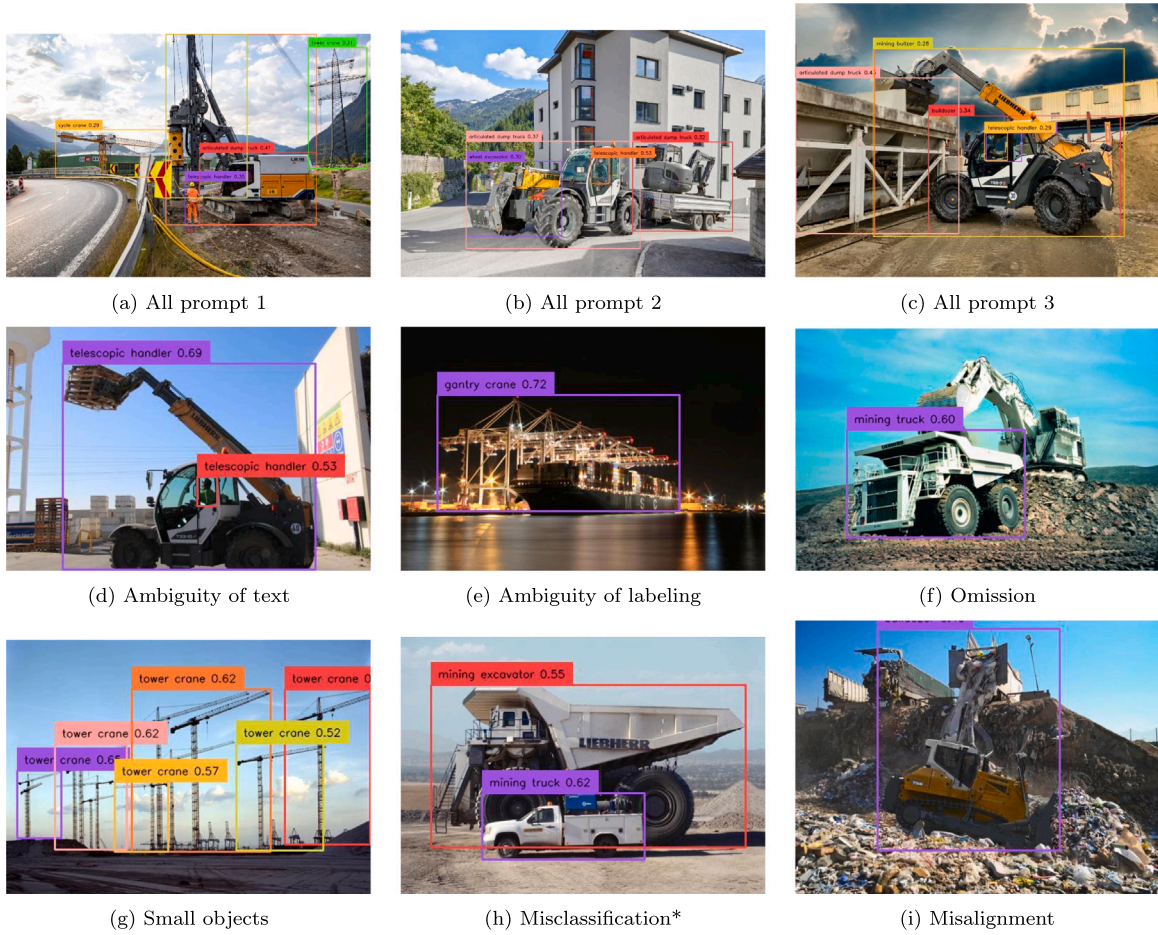


Fig. 12. Common data annotation errors by OVD (Grounding DINO). The first row is annotation results using all class names as a single text prompt. The second and third rows demonstrate common issues for OVD. The * denotes the only case when the issue is not discovered by GPT-4o-based pseudo-label review.

5. Discussion

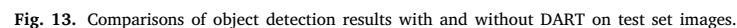
Limitation. While our DART pipeline demonstrates significant performance, boosting the AP of an already fine-tuned YOLOv8n model from 0.064 to 0.832 without any manual labeling, it is still constrained by weaknesses in the current instantiations of each module. Grounding DINO's issues are illustrated and discussed in Section 4.6. The Dream-booth with SDXL for data diversification generates objects that closely mimic originals in various poses and scenes but still suffer from slight dissimilarity and hallucination, which result in initially unrealistic generations and, eventually, the necessity of original data during training. For images disapproved by pseudo-label review, human intervention is still preferred to indiscriminate deletion, which leads to an inevitable loss of information. Compared to proprietary LMMs such as GPT-4o, open-source models like InternVL perform suboptimally and can only handle simple scenarios. A few categories show minimal improvement after diversification, primarily due to inadequate labeling of test set data and deficiencies noted above.

Future work. The modular design of our proposed DART system ensures both easy exchangeability and extensibility. This allows for future algorithm upgrades, the seamless integration of new object categories, and adaptability to customized environments without the need for manual labeling or additional data collection. Given that the current major bottleneck of DART is providing candidate ground truths through Grounding DINO, we hope future work will incorporate more robust OVD for annotations. The rapid advancement of Large Multimodal Models (LMMs) suggests that, in the near future, more powerful LMMs will be capable of effectively handling the review

process and even addressing the challenge of numerical coordinate generation. Nonetheless, we still recommend retaining both the annotation and review stages. Employing different model architectures for these tasks mirrors the roles of annotators and reviewers in manual labeling, ensuring maximum reliability of the ground truth. Although our experience indicates that hallucinations in generated images do not result in severe consequences, more realistic objects are undeniably beneficial. Finally, improved real-time object detectors will naturally boost the model's overall performance.

6. Conclusion

In this paper, we introduce **DART**, an automated end-to-end object detection pipeline to eliminate the need for human labeling and extensive data collection while maintaining high model performance. The Liebherr Product (LP) dataset with 15K images across 23 categories of construction machines is collected to validate the effectiveness of our approach. We have demonstrated the efficacy of the DART pipeline through extensive experimentation and analysis. Our results consistently show that incremental incorporation of each component of DART always contributes to significant improvements in AP. Detailed quantitative ablation studies, coupled with qualitative analyses, reveal the optimal configuration of each stage. Notably, our researches indicate that a 3:1 ratio of generated (by fine-tuned SDXL models via Dream-Booth) to original data yields the best results for data diversification on the LP dataset. Open-vocabulary bounding box annotation (Grounding DINO) has proven effective in generating credible bounding boxes without human intervention. GPT-4o has demonstrated human-level



customization for any specific target object or environment. Remarkably, DART accomplishes these results without requiring any manual annotation and extra data collection, further highlighting its significance. The efficiency and automation of DART not only streamline object detection applications but also reveal its potential to revolutionize the field.

Table A.7
Hyperparameters of Dreambooth with SDXL.

Hyperparameters	Values
pre-trained SDXL model	stabilityai/stable-diffusion-xl-base-1.0
pre-trained VAE model	madebyollin/sd-xl-vae-fp16-fix
optimizer	AdamW
β_1	0.9
β_2	0.999
weight decay	1×10^{-4}
mixed precision	BF16
resolution	1024×1024
batch size	1
learning rate for U-Net	1×10^{-4}
learning rate for text encoder	5×10^{-6}
learning rate schedule	constant
learning rate warmup steps	0
training steps	$\max(800, \{120, 140\} \times \text{\#images})$
gradient accumulation steps	1
prior preservation loss weight	1
SNR-gamma	5

CRedit authorship contribution statement

Chen Xin: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. **Andreas Hartel:** Resources, Project administration, Funding acquisition. **Enkelejda Kasneci:** Resources, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Appendix A. More model details

In this section, we discuss additional implementation details of the core modules of DART.

A.1. DreamBooth

The hyperparameters used for the Dreambooth with SDXL model are detailed in Table A.7. We use the official SDXL checkpoint from Stability AI but replace the VAE with madebyollin's version (for a fair comparison of FP16 and BF16, where we later find no apparent differences between the two) to initialize the SDXL model. The initialized SDXL model is then trained using LoRA. We choose the AdamW optimizer with a learning rate 1×10^{-4} for U-Net and 5×10^{-6} for the text encoder. The learning rate schedule is constant, with no warmup steps. We implement dynamic training steps by adjusting the maximum training steps based on the number of images collected for the corresponding instance. Specifically, two models are trained for each instance with maximum training steps set to either 120 or 140 times the actual number of images collected for the corresponding instance during preprocessing. For weighting, the prior preservation loss's weight is set to 1 and the SNR-gamma to 5 for all experiments.

Fig. A.14 illustrates the distribution of the number of instances by the number of images per instance. The minimum number of images per instance is set to 3 (as suggested by Ruiz et al., 2023), while the maximum is 16. We do not find a significant correlation between the number of images per instance and the quality of the trained SDXL via DreamBooth. Most instances contain approximately 10 images each, which we empirically find to be a suitable choice.

Table A.8
Hyperparameters of YOLOv8 and YOLOv10 models.

Hyperparameter	YOLOv8n/v8s/v10n/v10s
epochs	60
optimizer	AdamW
β_1	0.9
β_2	0.999
weight decay	5×10^{-4}
mixed precision	FP16
resolution	640×640
initial learning rate	$27/7.3/50/7.0 \times 10^{-5}$
final learning rate factor	0.5
batch size	64/32/32/32
learning rate schedule	cosine/cosine/cosine/linear
warm-up epochs	4/1/3/4
warm-up momentum	0.8
warm-up bias learning rate	0.1
box loss weight	7.5
class loss weight	0.5
DFL loss weight	1.5
HSV saturation	0.7
HSV value	0.4
HSV hue	0.015
translation	0.1
flip probability	0.5
scale	0.5
mosaic probability	1.0
close mosaic epochs	10

A.2. LMM

Besides GPT-4o, we have also tried GPT-4v with the “gpt-4-vision-preview” API and observed that GPT-4v is particularly strict when handling bounding boxes, which leads to the “relaxation” and “reminder” section of the user prompt listed in Table B.11. Additionally, we find that if the prompt is too lengthy, all LMM, except for GPT-4o, experience a decline in performance. Therefore, we deliberately shorten the prompt for InternVL-1.5, as shown in Table B.12. For GPT-4v, we discover a workaround for this issue: using multi-turn dialogues, where we first ask the questions (which corresponds to “task”, “question”, and “output” section in Table B.11) and then give suggestions (“relaxation” and “reminder” section in Table B.11) in the second turn. This approach helps avoid overly long prompts and improves model performance. In contrast, GPT-4o does not suffer from long prompts and dominates its predecessor in the bounding box review task. Therefore, we exclusively use it for the pseudo-label review segment of our DART pipeline.

A.3. YOLO

Table A.8 presents hyperparameters used for YOLO models. We follow common practice (Anon, 2020; Jocher, 2023; Li, Li et al., 2023; Li et al., 2022; Wang et al., 2023; Wang, Chen et al., 2024; Wang et al., 2024) to resize all images to 640×640 . In almost all experiments, training is conducted over 60 epochs since model performance plateaus after 20–30 epochs. AdamW optimizer is always employed with default beta settings and a weight decay set to 5×10^{-4} . For each experiment setup and model type, we meticulously fine-tune learning-rate-related parameters and only display the optimal hyperparameter sets derived from the experiment setup of the optimal generated-to-original data ratio (3:1) in Table A.8. It turns out that the optimal learning rates range from $7e-5$ to $5e-4$, with a general trend indicating that larger models and bigger datasets benefit from lower learning rates. The final learning rate factor is always 50% at best. The learning rate schedule is either cosine or linear, depending on the model. Warm-up epochs are set between 1 and 4, with a warm-up momentum of 0.8 and a warm-up bias learning rate of 0.1. The box loss weight is set to 7.5, class loss weight to 0.5, and DFL loss weight to 1.5. HSV augmentations are specified by saturation with 0.7, value with 0.4, and hue with 0.015. Mosaic augmentation is always applied (with probability 100%)

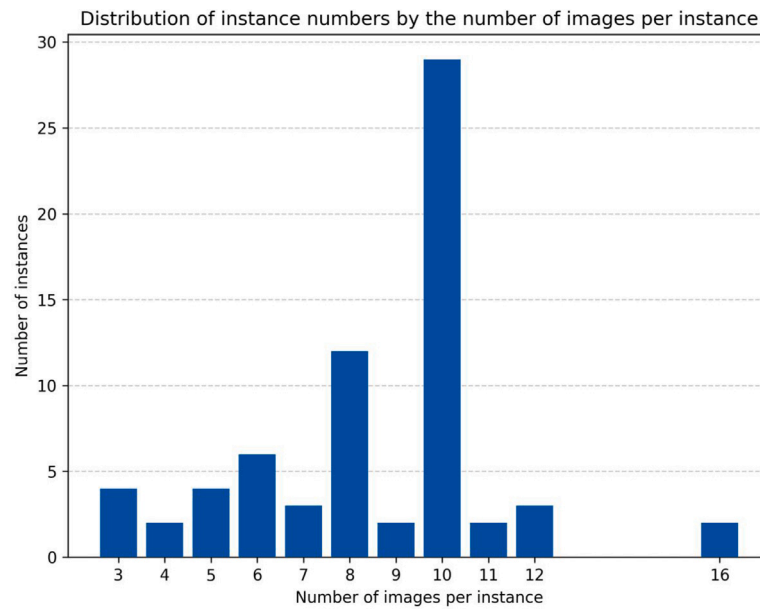


Fig. A.14. Distribution of instance numbers by the number of images per instance.

Table B.9

A list of all prompts used during data generation via trained SDXL by DreamBooth. The “{placeholder}” refers to the specific instance name wrapped by “<>” followed by its class name, e.g. “<TA230> articulated dump truck” for the case in Fig. 2.

ID	Description	Prompt text
1	construction site	A photo of a {placeholder} on a construction site. The image is high quality and photorealistic. The {placeholder} may be partially visible, at a distance, or obscured. The background is complex, providing a realistic context.
2	Sunny construction site	A high-quality, photorealistic image of a {placeholder} under a bright, sunny sky at a bustling construction site.
3	Cloudy construction site	A detailed, photorealistic image of a {placeholder} operating at a construction site on a cloudy day.
4	Rainy construction site	A high-resolution, photorealistic image of a {placeholder} working at a muddy construction site during heavy rain.
5	City street	A photorealistic image of a {placeholder} on a busy city street, surrounded by tall buildings and traffic.
6	Rural area	A high-quality, photorealistic image of a {placeholder} operating in a rural area, with fields and farmhouses in the background.
7	Mining site	A detailed, photorealistic image of a {placeholder} working at a mining site, with rocky terrain and machinery in the background.
8	Harbor	A high-resolution, photorealistic image of a {placeholder} operating at a harbor, with ships and cranes in the background.
9	Left side	A photorealistic image of a {placeholder} positioned on the left side of the frame at a construction site, with workers in the background.
10	Right side	A high-quality, photorealistic image of a {placeholder} on the right side of the image, operating in a busy urban environment.
11	In the distance	A detailed, photorealistic image of a {placeholder} visible in the distance at a construction site, with a panoramic view of the area.
12	Facing left	A high-resolution, photorealistic image of a {placeholder} facing left, working at a construction site with scaffolding in the background.
13	Facing right	A photorealistic image of a {placeholder} facing right, operating in a rural setting with rolling hills in the background.
14	Partially visible	A high-quality, photorealistic image of a {placeholder} partially visible behind a building at a construction site.
15	Nighttime construction	A photorealistic image of a {placeholder} working at night under artificial lights at a construction site.
16	Foggy morning	A detailed, photorealistic image of a {placeholder} operating on a foggy morning, with limited visibility at a rural construction site.
17	Snowy day	A high-resolution, photorealistic image of a {placeholder} working at a construction site during snowfall, with snow-covered equipment and ground.
18	Highway construction	A photorealistic image of a {placeholder} operating on a highway under construction, with traffic cones and barriers.
19	Desert construction	A high-quality, photorealistic image of a {placeholder} working at a construction site in a desert area, with sand dunes in the background.
20	Urban demolition	A high-resolution, photorealistic image of a {placeholder} demolishing an old building in an urban area, with debris flying.
21	Windy day	A high-quality, photorealistic image of a {placeholder} working at a construction site on a windy day, with dust and debris blowing in the wind.
22	Industrial area	A detailed, photorealistic image of a {placeholder} operating in an industrial area, surrounded by factories and heavy machinery.
23	Riverbank construction	A high-resolution, photorealistic image of a {placeholder} working on a riverbank, with water and vegetation in the background.

(continued on next page)

Table B.9 (continued).

ID	Description	Prompt text
24	Urban parking lot	A photorealistic image of a {placeholder} operating in an urban parking lot construction site, with buildings and cars nearby.
25	Dam construction	A high-quality, photorealistic image of {placeholder} working on a dam construction project, with water and concrete structures in the background.
26	Railway construction	A detailed, photorealistic image of a {placeholder} working on a railway construction site, with tracks and trains in the background.
27	Solar farm construction	A high-resolution, photorealistic image of a {placeholder} operating at a solar farm construction site, with solar panels and wide open fields.
28	Skyscraper construction	A photorealistic image of a {placeholder} working at a skyscraper construction site, with towering steel structures and cranes.
29	Wind farm construction	A high-quality, photorealistic image of a {placeholder} operating at a wind farm construction site, with wind turbines in the background.
30	Bridge construction	A detailed, photorealistic image of a {placeholder} working on a bridge construction site over a river.
31	Mountain construction	A high-quality, photorealistic image of a {placeholder} operating on a steep mountain construction site with rocky terrain.
32	Forest construction	A photorealistic image of a {placeholder} working in a forest area, with tall trees and underbrush surrounding the site.
33	Coastal construction	A detailed, photorealistic image of a {placeholder} operating at a coastal construction site, with the ocean and waves in the background.
34	Underground construction	A high-resolution, photorealistic image of a {placeholder} working in an underground tunnel construction site, with dim lighting and rocky walls.
35	Large scale construction	A high-quality, photorealistic image of multiple {placeholder}s working together on a large-scale construction project, with cranes and scaffolding.
36	Airport construction	A photorealistic image of a {placeholder} operating at an airport construction site, with planes and runways in the background.
37	Suburban construction	A detailed, photorealistic image of a {placeholder} working in a suburban area, with houses and trees around the construction site.
38	Island construction	A high-resolution, photorealistic image of a {placeholder} operating on an island construction site, surrounded by water and palm trees.
39	Urban renewal	A high-quality, photorealistic image of a {placeholder} working on an urban renewal project, surrounded by modern buildings and greenery.
40	Industrial site	A photorealistic image of a {placeholder} operating at an industrial construction site, with factories and smokestacks in the background.
41	Multiple machines sunny	A high-quality, photorealistic image of several {placeholder}s working together under a bright, sunny sky at a bustling construction site.
42	Multiple machines rainy	A high-resolution, photorealistic image of several {placeholder}s operating at a muddy construction site during heavy rain.
43	Multiple machines city street	A photorealistic image of multiple {placeholder}s on a busy city street, surrounded by tall buildings and traffic.
44	Multiple machines rural area	A high-quality, photorealistic image of several {placeholder}s operating in a rural area, with fields and farmhouses in the background.
45	Multiple machines harbor	A high-resolution, photorealistic image of several {placeholder}s operating at a harbor, with ships and cranes in the background.
46	Multiple machines night	A photorealistic image of several {placeholder}s working at night under artificial lights at a construction site.
47	Multiple machines snowy day	A high-resolution, photorealistic image of several {placeholder}s working at a construction site during snowfall, with snow-covered equipment and ground.
48	Multiple machines highway	A photorealistic image of several {placeholder}s operating on a highway under construction, with traffic cones and barriers.
49	Open pit mine	A high-quality, photorealistic image of a {placeholder} working in an open-pit mine, with terraced rock formations and heavy machinery around.
50	Underground mine	A detailed, photorealistic image of a {placeholder} operating in an underground mine, with tunnels and mining equipment.
51	Mining town	A high-resolution, photorealistic image of a {placeholder} working near a mining town, with workers' housing and facilities in the background.
52	Quarry operation	A photorealistic image of a {placeholder} working in a quarry, with large stone blocks and cutting equipment.
53	Mountain mining	A high-quality, photorealistic image of a {placeholder} operating in a mountain mining site, with steep slopes and rocky terrain.
54	Remote mine	A photorealistic image of a {placeholder} working in a remote mining location, with rugged terrain and minimal infrastructure.
55	Ore processing	A high-quality, photorealistic image of a {placeholder} involved in ore processing, with conveyor belts and sorting equipment.
56	Tailings dam	A detailed, photorealistic image of a {placeholder} working near a tailings dam, with water and waste materials.
57	Mining reclamation	A high-resolution, photorealistic image of a {placeholder} involved in mining reclamation, with land restoration and vegetation.
58	Ocean dockyard	A high-quality, photorealistic image of a {placeholder} operating in an ocean dockyard, with large ships and containers in the background.
59	River dredging	A detailed, photorealistic image of a {placeholder} dredging a river, with muddy water and vegetation along the banks.
60	Coastal maintenance	A high-resolution, photorealistic image of a {placeholder} performing coastal maintenance, with waves crashing in the background.
61	Offshore construction	A photorealistic image of a {placeholder} working on an offshore construction project, with deep sea and platform structures.
62	Flood control	A high-quality, photorealistic image of a {placeholder} engaged in flood control operations, with sandbags and rising water levels.
63	Canal work	A detailed, photorealistic image of a {placeholder} working on a canal, with narrow waterways and boats passing by.

(continued on next page)

Table B.9 (continued).

ID	Description	Prompt text
64	Shipyard repair	A photorealistic image of a {placeholder} in a shipyard, repairing large vessels with workers around.
65	Waterfront park	A high-quality, photorealistic image of a {placeholder} operating in a waterfront park, with recreational areas and people nearby.
66	Marine research	A detailed, photorealistic image of a {placeholder} assisting in marine research activities, with scientists and research equipment around.
67	Dockside assembly	A photorealistic image of a {placeholder} involved in dockside assembly of marine structures, with cranes and construction materials.

Table B.10

A list of synonyms (including superclasses) for each class in LP dataset.

Original class name	Synonyms
articulated dump truck	N/A
bulldozer	dozer, crawler tractor
crawler crane	track crane, crane
crawler excavator	track excavator, excavator
crawler loader	track loader
combined piling and drilling rig	drilling rig, piling rig
duty cycle crane	dragline, dragline excavator
gantry crane	container crane, maritime crane
log loader	log handling machine
maritime crane	harbor crane, port crane, crane
material handling machine	material handler
mining bulldozer	N/A
mining excavator	N/A
mining truck	N/A
mobile crane	crane
pipelayer	sideboom
pontoon excavator	floating excavator, amphibious excavator, excavator
reachstacker	container handler, material handling equipment
telescopic handler	lull, telehandler, reach forklift, zoom boom, material handling equipment
tower crane	crane
truck mixer	cement mixer truck, concrete mixer truck
wheel loader	front end loader, bucket loader
wheel excavator	mobile excavator, excavator

except for the last 10 epochs when it is turned off for better alignment. Additional parameters include translation at 0.1 fraction of total width and height, horizontal flip rate at 0.5, and scale at 0.5 (resizing between 50% and 150%).

Appendix B. Prompt engineering

Prompt engineering is critical for the performance of three out of four major components of our proposed DART pipeline. In this section, we provide details about how the prompts are constructed for each component.

B.1. DreamBooth

Table B.9 displays all the prompts used for inference in DreamBooth with SDXL to diversify our dataset. For each prompt, we provide an overall description, the exact prompt text, and their respective IDs. Prompts 1–48 are general prompts applied across all classes. Prompts 49–57 are specific to large land-based machinery, such as mining equipment, while prompts 58–67 are tailored for water-related machines, such as maritime cranes.

B.2. Grounding DINO

Table B.10 presents the synonyms (and superclasses, combined as one column) for each category in the LP dataset. The synonyms are used to construct specific prompts during the annotation phase. The detailed procedure is described in Section 3.3.

B.3. LMM

GPT-4o receives detailed prompts for bounding box review. The prompts are broken into parts in Table B.11, with the purposes given on the left and the exact text contents given on the right. First, The system prompt briefly summarizes the bounding box review task and highlights three key metrics: precision, recall, and fit. The user prompt consists of the five remaining sections. The “task” section outlines the task and objectives in more detail, with {target} referring to the pre-defined class during data collection and {secondary_target} indicating commonly co-occurring classes also recorded during data collection. Note that explicit mention of target types is not mandatory since the boxes in the images already have class labels, and GPT-4o exhibits excellent OCR capability. But it is still beneficial for corner cases where the label in the image is occluded. Next, the “questions” section elaborates on the specific questions corresponding to the three main criteria. The output format is specified in the “output” segment to facilitate subsequent processing. The “output” section also follows the guidelines of COT to enhance the model’s performance. In our early experiments, we found that GPT-4 models tend to be overly strict in their judgments regarding the bounding box. To mitigate this, we summarize common issues in the “relaxation” and “reminder” sections and attach them to the end of the user prompt. Additionally, our tests with GPT-4v show that including the “relaxation” and “reminder” sections as part of the second round of dialogue instead of a single long text is more effective. However, GPT-4o can handle all sections of the user prompt at once.

Table B.11

Text prompt for GPT-4o-based bounding box review. The “system” prompt in the table is used as the system prompt for the API, while the user prompt is created by sequentially concatenating the remaining parts.

Description	Prompt text
system	You are an AI bounding box annotation evaluator. Your task is to evaluate the correctness of bounding box annotations for given images and target objects. The bounding boxes are directly drawn as colored rectangles on top of the image. The class label is shown at the top-left corner of the corresponding bounding box. You will evaluate each bounding box annotation based on three criteria: precision, recall, and fit.
task	In this image, the target object for bounding box annotation is {target}. There may also be {secondary_target}. Each of existing {target}s and {secondary_target}s should be annotated by a bounding box drawn as a colored rectangle. The goal is to accurately localize all {target}s and {secondary_target}s using these bounding boxes. Your task is to evaluate whether all bounding boxes are correct.
questions	Correctness should be assessed in terms of precision, recall, and fit. Specifically, consider the following questions before making your judgment: 1. Does each bounding box perfectly enclose one single target object? 2. Are all target objects localized by a bounding box? 3. Is each bounding box neither too loose nor too tight?
output	Please provide your evaluation in the following JSON format: “json”: “Precision”: “Yes/No answer to question 1”, “Recall”: “Yes/No answer to question 2”, “Fit”: “Yes/No answer to question 3” } Please think step-by-step and be sure to provide the correct answers. Very briefly explain yourself before answering the question.
relaxation	Before finalizing your evaluation, please consider the following suggestions: 1. For question 1 (Precision), if an object is occluded, the bounding box should be inferred based on a reasonable estimation of the object’s size. 2. For question 2 (Recall), it is fairly normal to have only one object in the dataset. 3. For question 3 (Fit), don’t be too harsh when bounding box edges just slightly cut off the object or just enclose a little bit of the outside area.
reminder	Always consider my suggestions before answering questions. But the suggestions are not strict rules. You can also use your own judgment. The most important thing is to answer the three questions correctly.

Table B.12

Text prompt for InternVL-1.5-based image photorealism review. The “{target}” is a placeholder for the actual class name.

Description	Prompt text
User	Is this image suitable as training data object detection for {target}? Answer YES or NO. Does the main object in the image look like an authentic {target}? Answer YES or NO.

For InternVL-1.5, concise prompts are used to review the photorealism of a generated image, avoiding unnecessary complexity and the performance degradation issue for long text. There is only a single prompt used for each image, which is listed in Table B.12. Like the bounding box review, we only keep images receiving positive feedback for all questions from InternVL-1.5.

Appendix C. More visualizations

In this section, we provide additional visualization results for qualitative analysis of the DART pipeline.

We provide additional visual comparisons of object detection results with and without DART in Fig. C.15. Same as in Fig. 13, the implementation of DART leads to a significant increase in model performance.

Fig. C.16 provides more examples for visualization of data diversification and bounding box annotation detailed in Section 4.6.

Figs. C.17, C.18, and C.19 showcase the effectiveness of LMM-based review. Specifically, Figs. C.17 and C.18 illustrate annotated images approved and disapproved by GPT-4o during pseudo-label review, respectively. GPT-4o not only accurately assesses bounding box annotations in complex scenarios, including multiple objects and occlusion, but also identifies common annotation errors, such as missing objects, wrong labels, and inaccurate bounding box coordinates. On the other hand, the results of the photorealism review of generated images are demonstrated in Fig. C.19. InternVL-1.5 can detect unrealistic objects and scenes. Preventing these inferiors from being included in the dataset is proven (in Sections 4.4 and 4.5.3) to help improve the performance. The cases provided by these figures further validate the efficacy and essential role of the LMM-based review in the DART pipeline.

Finally, Fig. C.20 illustrates comparisons between SDXL and SD-1.5 for data diversification. The two rows of generated images on the right side of the figure are produced by SD-1.5 and SDXL, respectively. To ensure a fair comparison, both models are trained under the same DreamBooth framework by identical instance data, and the images are induced by the exact same prompts (with ID and description given as

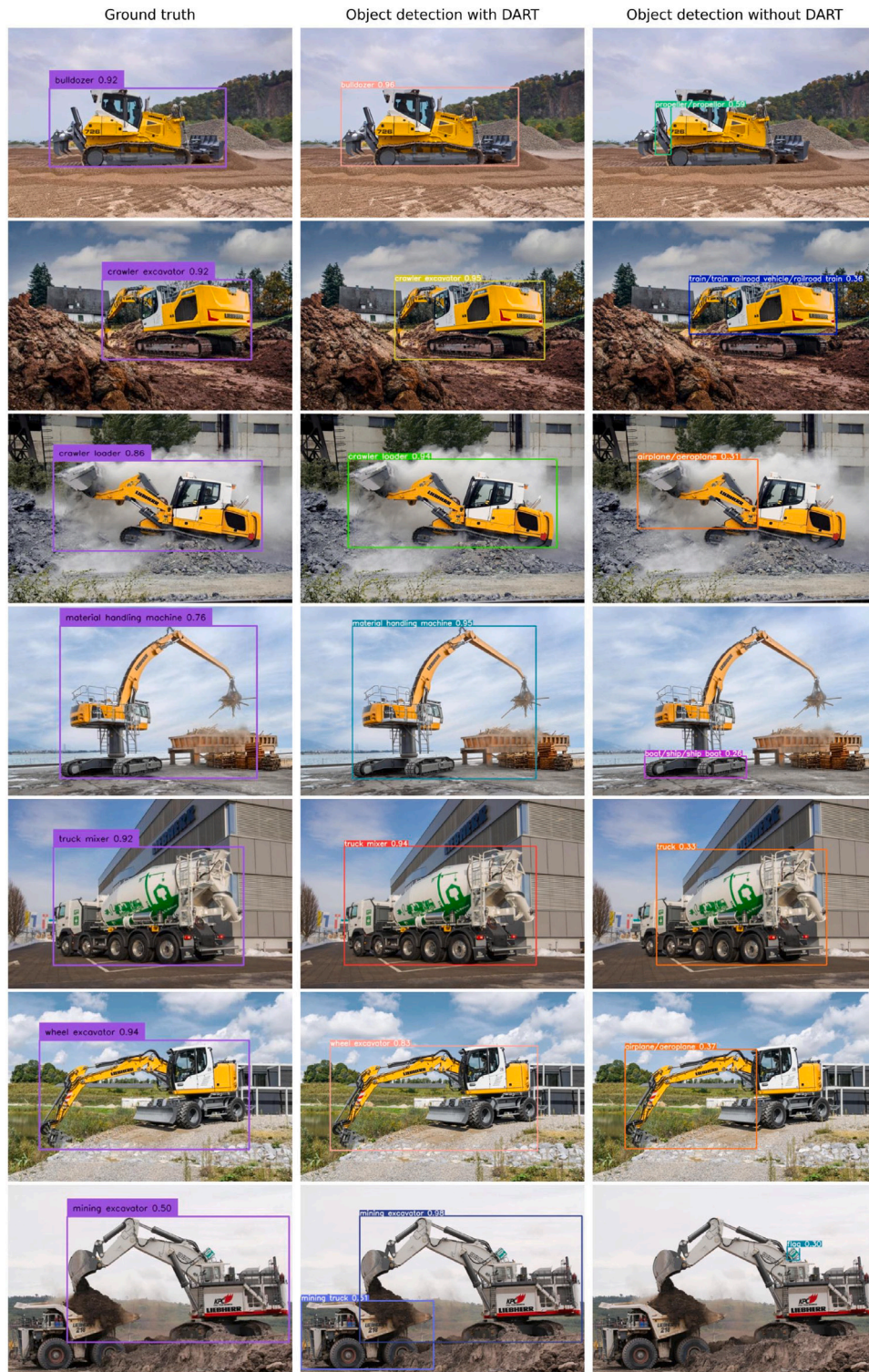


Fig. C.15. More comparisons of object detection results with and without DART on test set images.

captions underneath; refer to Table B.9 for the exact text prompt) during inference. It is clear from the figure that images generated by SDXL are not only more detailed and realistic but also better at interpreting and following complex text prompts. In contrast, SD-1.5 images tend

to have monotonous poses and simple scenes. More importantly, SD-1.5 fails to capture the diverse scenarios described in the text prompts, especially for unusual scenes. Therefore, we only adopt DreamBooth with SDXL for our data diversification phase.

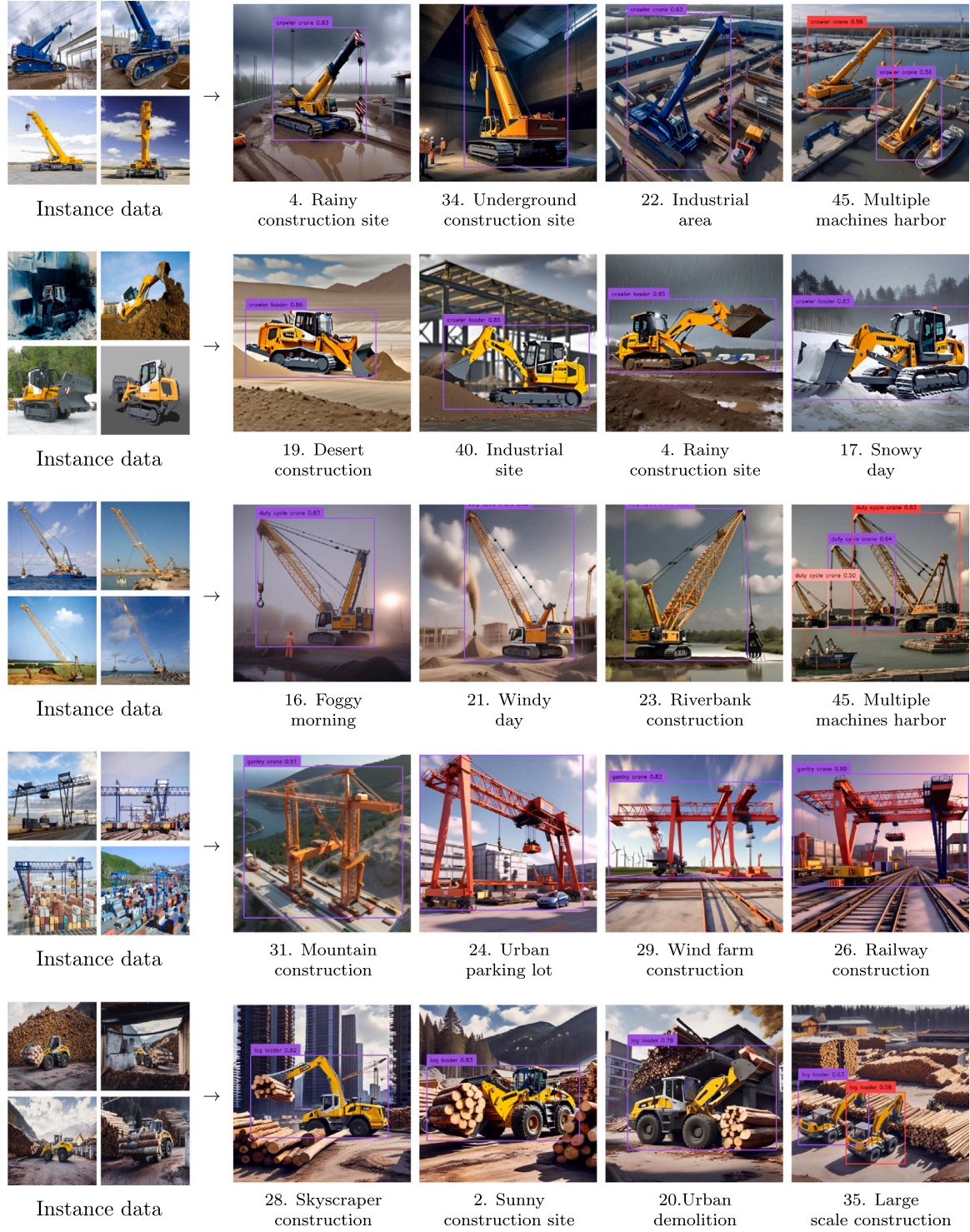


Fig. C.16. Visualization of data diversification and bounding box annotation for additional categories.

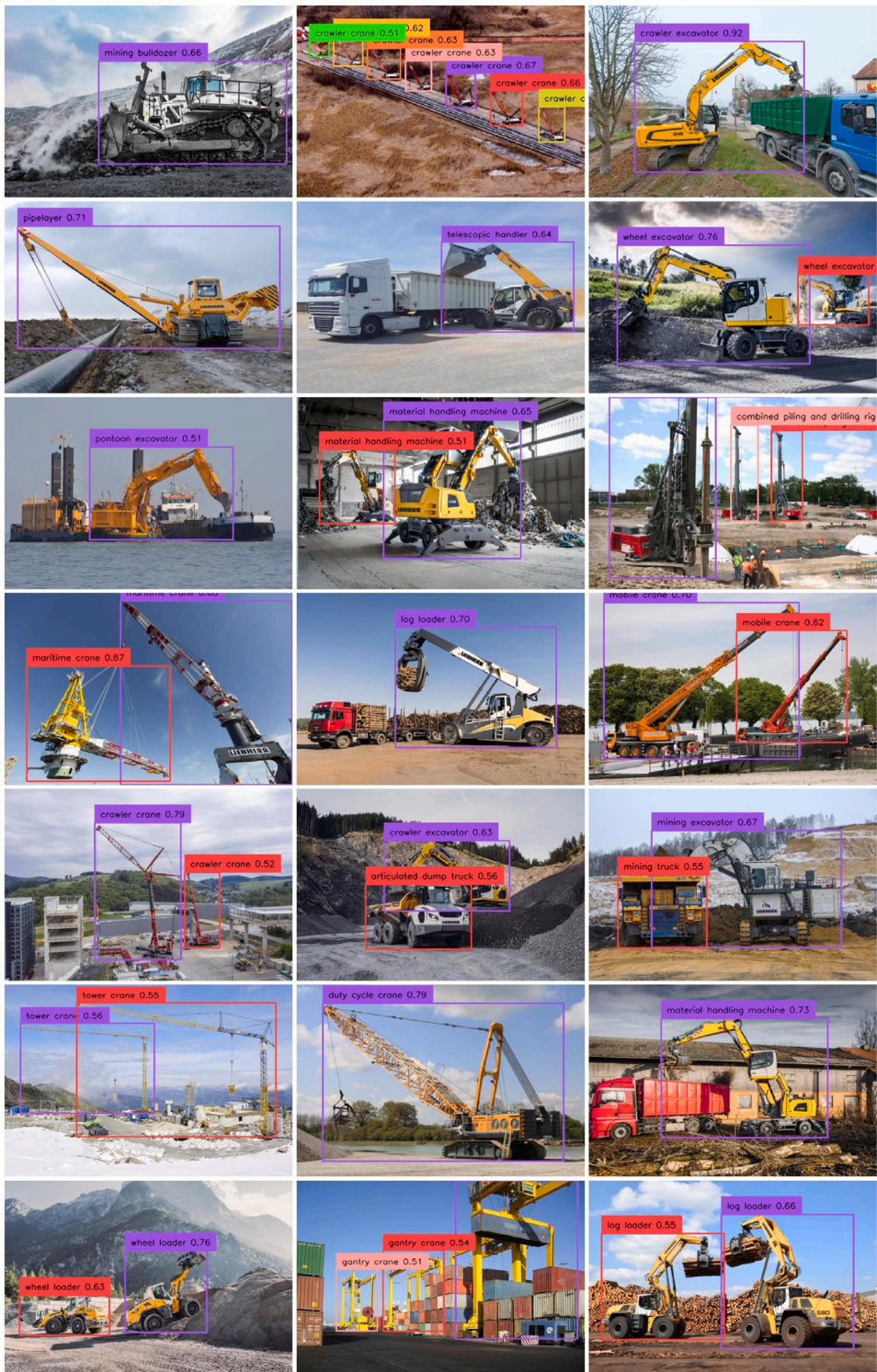


Fig. C.17. Original images annotated by Grounding DINO and approved by GPT-4o.

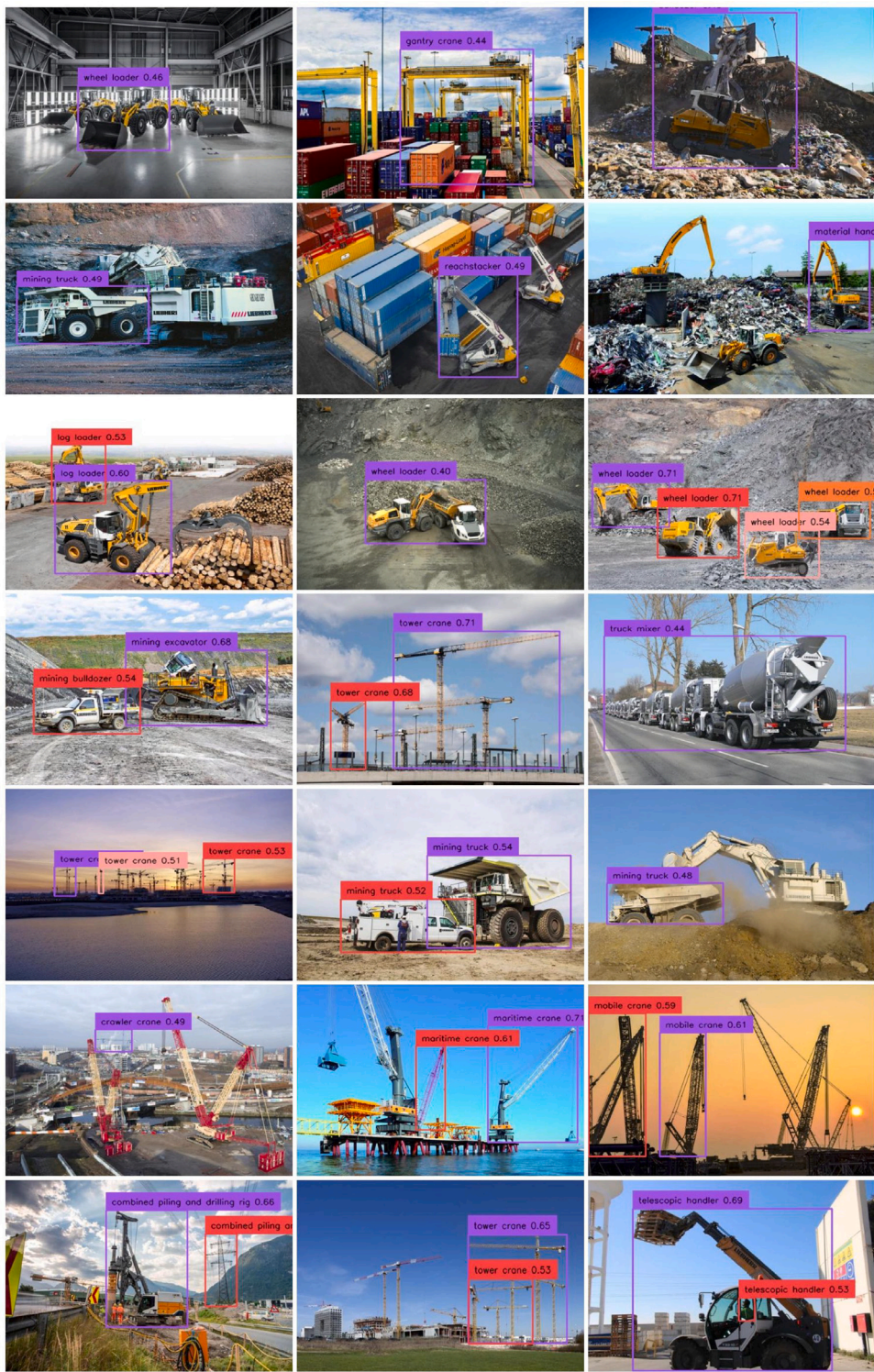


Fig. C.18. Original images annotated by Grounding DINO but disapproved by GPT-4o.

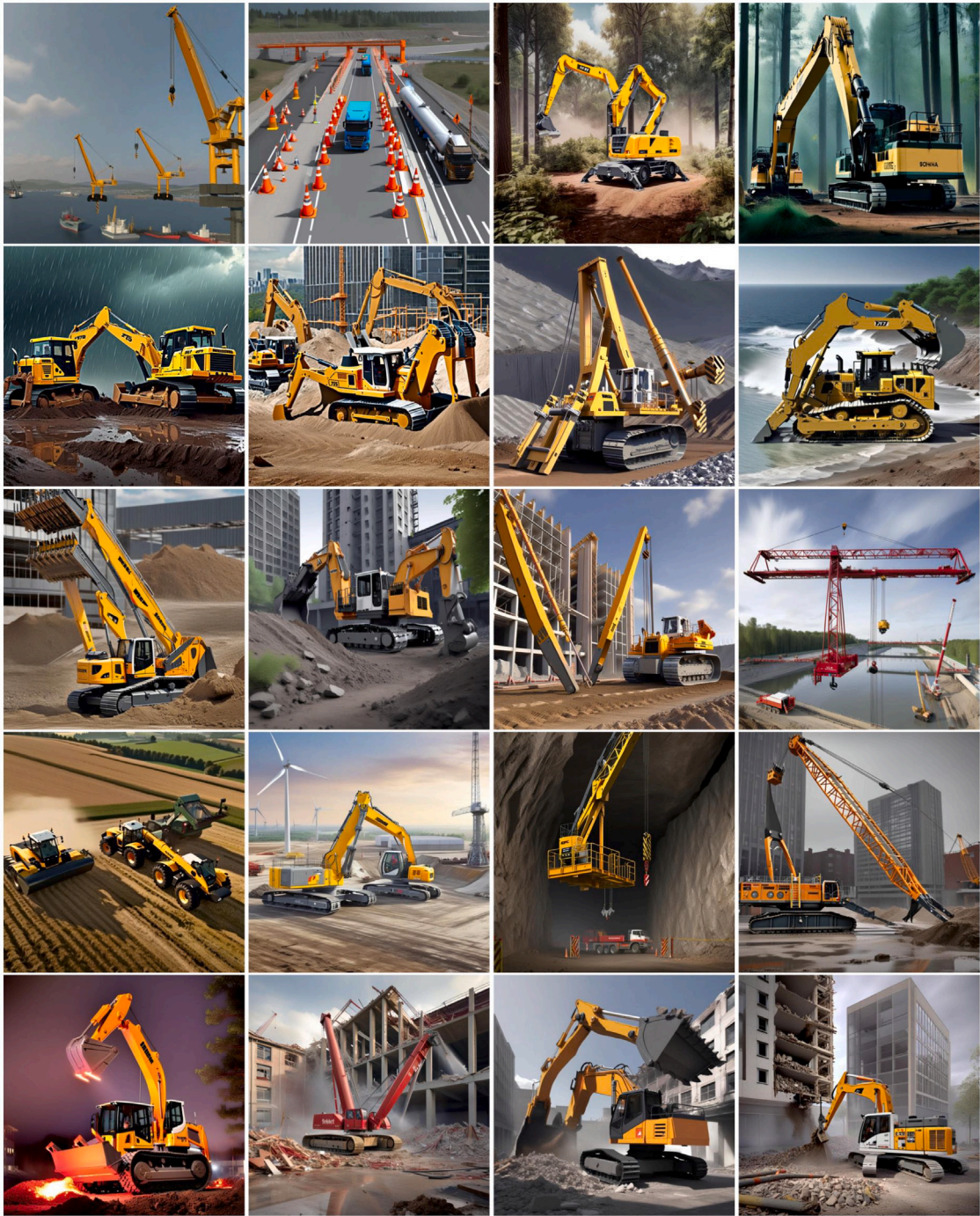


Fig. C.19. Generated images by DreamBooth with SDXL but disapproved by InternVL-1.5.

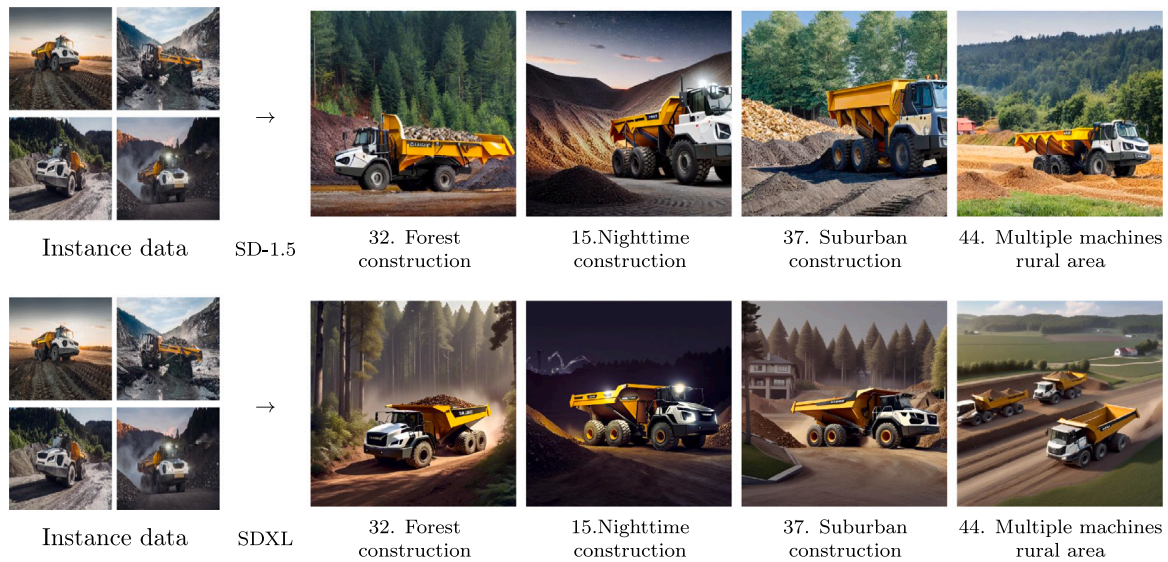


Fig. C.20. SDXL vs SD-1.5.

References

- Alayrac, J.-B., Donahue, J., Luc, P., Miech, A., Barr, I., Hasson, Y., et al. (2022). Flamingo: A visual language model for few-shot learning. <http://dx.doi.org/10.48550/arXiv.2204.14198>, arXiv:2204.14198.
- Anon (2020). YOLOv5 by ultralytics. URL <https://github.com/ultralytics/yolov5>.
- Anthropic (2024). Claude 3 family. URL <https://www.anthropic.com/news/claude-3-family>.
- Bochkovskiy, A., Wang, C.-Y., & Liao, H.-Y. M. (2020). YOLOv4: Optimal speed and accuracy of object detection. <http://dx.doi.org/10.48550/arXiv.2004.10934>, arXiv:2004.10934.
- Chen, Z., Wang, W., Tian, H., Ye, S., Gao, Z., Cui, E., et al. (2024). How far are we to GPT-4V? closing the gap to commercial multimodal models with open-source suites. <http://dx.doi.org/10.48550/arXiv.2404.16821>, arXiv:2404.16821.
- Chen, Z., Wu, J., Wang, W., Su, W., Chen, G., Xing, S., et al. (2024). InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. <http://dx.doi.org/10.48550/arXiv.2312.14238>, arXiv:2312.14238.
- Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X., & Shan, Y. (2024). YOLO-world: Real-time open-vocabulary object detection. In *Proc. IEEE conf. computer vision and pattern recognition*.
- Cho, J., & Kim, K. (2023). Detection of moving objects in multi-complex environments using selective attention networks (SANet). *Automation in Construction*, 155, Article 105066. <http://dx.doi.org/10.1016/j.autcon.2023.105066>.
- Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., & Sun, J. (2021). Repvgg: Making vgg-style convnets great again. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 13733–13742).
- Dong, C., Jiang, X., Hu, Y., Du, Y., & Pan, L. (2024). EL-Net: An efficient and lightweight optimized network for object detection in remote sensing images. *Expert Systems with Applications*, 255, Article 124661. <http://dx.doi.org/10.1016/j.eswa.2024.124661>.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., et al. (2021). An image is worth 16x16 words: transformers for image recognition at scale. <http://dx.doi.org/10.48550/arXiv.2010.11929>, arXiv:2010.11929, [cs].
- Feng, C., Zhong, Y., Gao, Y., Scott, M. R., & Huang, W. (2021). TOOD: Task-aligned one-stage object detection. In *ICCV*.
- Gal, R., Alaluf, Y., Atzmon, Y., Patashnik, O., Bermano, A. H., Chechik, G., et al. (2022). An image is worth one word: personalizing text-to-image generation using textual inversion. <http://dx.doi.org/10.48550/arXiv.2208.01618>, arXiv:2208.01618.
- Gao, J., Zhang, Y., Geng, X., Tang, H., & Bhatti, U. A. (2024). PE-Transformer: Path enhanced transformer for improving underwater object detection. *Expert Systems with Applications*, 246, Article 123253. <http://dx.doi.org/10.1016/j.eswa.2024.123253>.
- Gevorgyan, Z. (2022). SiO Loss: More powerful learning for bounding box regression. arXiv:2205.12740, URL <http://arxiv.org/abs/2205.12740>.
- Google, G. T. (2024). Gemini: A family of highly capable multimodal models. arXiv:2312.11805, URL <http://arxiv.org/abs/2312.11805>.
- Gu, X., Lin, T.-Y., Kuo, W., & Cui, Y. (2022). Open-vocabulary object detection via vision and language knowledge distillation. In *International conference on learning representations*. URL <https://openreview.net/forum?id=IL3lnMBr4WU>.
- Gupta, A., Dollar, P., & Girshick, R. (2019). Lvis: A dataset for large vocabulary instance segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 5356–5364).
- Hang, T., Gu, S., Li, C., Bao, J., Chen, D., Hu, H., et al. (2023). Efficient diffusion training via min-SNR weighting strategy. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 7441–7451).
- He, K., Zhang, X., Ren, S., & Sun, J. (2014). Spatial pyramid pooling in deep convolutional networks for visual recognition. In D. Fleet, T. Pajdla, B. Schiele, & T. Tuytelaars (Eds.), *Computer vision – ECCV 2014* (pp. 346–361). Cham: Springer International Publishing.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., et al. (2022). LoRA: Low-rank adaptation of large language models. In *International conference on learning representations*. URL <https://openreview.net/forum?id=nZeVKeeFYf9>.
- Jocher, G. (2023). Ultralytics YOLOv8. URL <https://github.com/ultralytics/ultralytics>.
- Klinger, E., & Starkweather, D. (2008). pHash.org: Home of pHash, the open source perceptual hash library. URL <http://www.phash.org/>.
- Lee, G.-G., & Zhai, X. (2023). NERIF: GPT-4V for automatic scoring of drawn models. <http://dx.doi.org/10.48550/arXiv.2311.12990>, arXiv:2311.12990.
- Li, C., Li, L., Geng, Y., Jiang, H., Cheng, M., Zhang, B., et al. (2023). YOLOv6 v3.0: A full-scale reloading. <http://dx.doi.org/10.48550/arXiv.2301.05586>, arXiv:2301.05586.
- Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., et al. (2022). YOLOv6: A single-stage object detection framework for industrial applications. arXiv:2209.02976, URL <http://arxiv.org/abs/2209.02976>.
- Li, X., Wang, W., Wu, L., Chen, S., Hu, X., Li, J., et al. (2020). Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection. In *NeurIPS*.
- Li, C., Wong, C., Zhang, S., Usuyama, N., Liu, H., Yang, J., et al. (2023). LLaVA-med: Training a large language-and-vision assistant for biomedicine in one day. In *Thirty-seventh conference on neural information processing systems datasets and benchmarks track*. URL <https://openreview.net/forum?id=GSuP99u2kR>.
- Li*, L. H., Zhang*, P., Zhang*, H., Yang, J., Li, C., Zhong, Y., et al. (2022). Grounded language-image pre-training. In *CVPR*.
- Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., et al. (2014). Microsoft COCO: Common objects in context. In *ECCV (ECCV)*. European Conference on Computer Vision, URL <https://www.microsoft.com/en-us/research/publication/microsoft-coco-common-objects-in-context/>.
- Liu, H., Li, C., Li, Y., & Lee, Y. J. (2024). Improved baselines with visual instruction tuning. <http://dx.doi.org/10.48550/arXiv.2310.03744>, arXiv:2310.03744.
- Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). Visual instruction tuning.
- Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., et al. (2021). Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 10012–10022).
- Liu, S., Qi, L., Qin, H., Shi, J., & Jia, J. (2018). Path aggregation network for instance segmentation. <http://dx.doi.org/10.48550/arXiv.1803.01534>, arXiv:1803.01534.
- Liu, S., Zeng, Z., Ren, T., Li, F., Zhang, H., Yang, J., et al. (2023). Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. <http://dx.doi.org/10.48550/arXiv.2303.05499>, arXiv:2303.05499.
- Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International conference on learning representations*. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Mou, C., Wang, X., Xie, L., Wu, Y., Zhang, J., Qi, Z., et al. (2023). T2I-Adapter: Learning adapters to dig out more controllable ability for text-to-image diffusion models. <http://dx.doi.org/10.48550/arXiv.2302.08453>, arXiv:2302.08453.

- OpenAI (2023). GPT-4 technical report. <http://dx.doi.org/10.48550/arXiv.2303.08774>, arXiv:2303.08774.
- OpenAI (2024). GPT-4o. URL <https://openai.com/index/hello-gpt-4o/>.
- Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., et al. (2024). SDXL: Improving latent diffusion models for high-resolution image synthesis. In *The twelfth international conference on learning representations*. URL <https://openreview.net/forum?id=di52zR8xgf>.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., et al. (2021). Learning transferable visual models from natural language supervision. In *International conference on machine learning* (pp. 8748–8763). PMLR.
- Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). Hierarchical text-conditional image generation with CLIP latents. arXiv:2204.06125, URL <http://arxiv.org/abs/2204.06125>.
- RangeKing (2023). Brief summary of YOLOv8 model structure. URL <https://github.com/ultralytics/ultralytics/issues/189>.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: unified, real-time object detection. In *CVPR*.
- Redmon, J., & Farhadi, A. (2017). YOLO9000: Better, faster, stronger. In *CVPR*.
- Redmon, J., & Farhadi, A. (2018). YOLOv3: An incremental improvement. <http://dx.doi.org/10.48550/arXiv.1804.02767>, arXiv:1804.02767.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 conference on empirical methods in natural language processing*. Association for Computational Linguistics, URL <http://arxiv.org/abs/1908.10084>.
- Ren, S., He, K., Girshick, R., & Sun, J. (2015). Faster R-CNN: Towards real-time object detection with region proposal networks. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, & R. Garnett (Eds.), *Vol. 28, Advances in neural information processing systems*. Curran Associates, Inc., URL https://proceedings.neurips.cc/paper_files/paper/2015/file/14bfa6bb14875e45bba028a21ed38046-Paper.pdf.
- Rezatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., & Savarese, S. (2019). Generalized intersection over union: a metric and A loss for bounding box regression. In *CVPR*.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10684–10695).
- Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. <http://dx.doi.org/10.48550/arXiv.1505.04597>, arXiv:1505.04597.
- Ruiz, N., Li, Y., Jampani, V., Pritch, Y., Rubinstein, M., & Aberman, K. (2023). Dream-Booth: Fine tuning text-to-image diffusion models for subject-driven generation. <http://dx.doi.org/10.48550/arXiv.2208.12242>, arXiv:2208.12242.
- Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E., et al. (2022a). Photorealistic text-to-image diffusion models with deep language understanding. <http://dx.doi.org/10.48550/arXiv.2205.11487>, arXiv:2205.11487.
- Saharia, C., Chan, W., Saxena, S., Li, L., Whang, J., Denton, E. L., et al. (2022b). Photorealistic text-to-image diffusion models with deep language understanding. *Advances in Neural Information Processing Systems*, 35, 36479–36494.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.
- Wang, Q., Bai, X., Wang, H., Qin, Z., Chen, A., Li, H., et al. (2024). InstantID: Zero-shot identity-preserving generation in seconds. arXiv:2401.07519, URL <http://arxiv.org/abs/2401.07519>.
- Wang, C.-Y., Bochkovskiy, A., & Liao, H.-Y. M. (2023). YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*.
- Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., et al. (2024). YOLOv10: Real-time end-to-end object detection. <http://dx.doi.org/10.48550/arXiv.2405.14458>, arXiv:2405.14458.
- Wang, W., Dai, J., Chen, Z., Huang, Z., Li, Z., Zhu, X., et al. (2023). InternImage: Exploring large-scale vision foundation models with deformable convolutions. <http://dx.doi.org/10.48550/arXiv.2211.05778>, arXiv:2211.05778.
- Wang, C.-Y., Liao, H.-Y. M., & Yeh, I.-H. (2022). Designing network design strategies through gradient path analysis. arXiv:2211.04800, URL <http://arxiv.org/abs/2211.04800>.
- Wang, C.-Y., Mark Liao, H.-Y., Wu, Y.-H., Chen, P.-Y., Hsieh, J.-W., & Yeh, I.-H. (2020). CSPNet: A new backbone that can enhance learning capability of cnn. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops* (pp. 390–391).
- Wang, C.-Y., Yeh, I.-H., & Liao, H.-Y. M. (2024). YOLOv9: Learning what you want to learn using programmable gradient information. <http://dx.doi.org/10.48550/arXiv.2402.13616>, arXiv:2402.13616.
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., et al. (2023). Chain-of-thought prompting elicits reasoning in large language models. <http://dx.doi.org/10.48550/arXiv.2201.11903>, arXiv:2201.11903.
- Xu, Z., Jha, N., Mehadi, S., & Mandal, M. (2024). Multiscale object detection on complex architectural floor plans. *Automation in Construction*, 165, Article 105486. <http://dx.doi.org/10.1016/j.autcon.2024.105486>.
- Xu, R., Yao, Y., Guo, Z., Cui, J., Ni, Z., Ge, C., et al. (2024). LLaVA-UHD: An LMM perceiving any aspect ratio and high-resolution images. <http://dx.doi.org/10.48550/arXiv.2403.11703>, arXiv:2403.11703.
- Yang, Z., Li, L., Lin, K., Wang, J., Lin, C.-C., Liu, Z., et al. (2023). The Dawn of LMMs: preliminary explorations with GPT-4V(ision). <http://dx.doi.org/10.48550/arXiv.2309.17421>, arXiv:2309.17421.
- Yang, J., Zhang, H., Li, F., Zou, X., Li, C., & Gao, J. (2023). Set-of-mark prompting unleashes extraordinary visual grounding in GPT-4V. <http://dx.doi.org/10.48550/arXiv.2310.11441>, arXiv:2310.11441.
- Ye, H., Zhang, J., Liu, S., Han, X., & Yang, W. (2023). IP-Adapter: Text compatible image prompt adapter for text-to-image diffusion models. <http://dx.doi.org/10.48550/arXiv.2308.06721>, arXiv:2308.06721.
- Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., & Ren, D. (2020). Distance-IoU loss: Faster and better learning for bounding box regression. In *The AAAI conference on artificial intelligence* (pp. 12993–13000).
- Zhu, D., Chen, J., Shen, X., Li, X., & Elhoseiny, M. (2023). MiniGPT-4: Enhancing vision-language understanding with advanced large language models. <http://dx.doi.org/10.48550/arXiv.2304.10592>, arXiv:2304.10592.
- Zong, Z., Song, G., & Liu, Y. (2023). DETRs with collaborative hybrid assignments training. arXiv:2211.12860, URL <http://arxiv.org/abs/2211.12860>.